

我是 TOC

GYF

2023 年 12 月

计算理论基础复习提要

李元《计算理论基础》复习提要。

制作者：GYF

GitHub: <https://github.com/23300240002>

第一节课复习资料

一、本节课知识点讲解

1.1 渐近记号 (Asymptotic Notation)

渐近记号用于描述函数在自变量趋于无穷时的增长趋势，是分析算法时间复杂度的核心工具。

大 O 记号 (Big-O) 定义: $f(n) = O(g(n))$ 表示存在正常数 c 和 n_0 , 使得对所有 $n \geq n_0$ 有 $|f(n)| \leq c|g(n)|$ 。

含义: $f(n)$ 的增长速度不超过 $g(n)$ 的常数倍, 描述上界。

例子: $3n^2 + 2n + 1 = O(n^2)$ 。

Ω 记号 (Omega) 定义: $f(n) = \Omega(g(n))$ 表示存在正常数 c 和 n_0 , 使得对所有 $n \geq n_0$ 有 $|f(n)| \geq c|g(n)|$ 。

含义: 描述下界。

Θ 记号 (Theta) 定义: $f(n) = \Theta(g(n))$ 表示同时有 $f(n) = O(g(n))$ 和 $f(n) = \Omega(g(n))$ 。

含义: $f(n)$ 与 $g(n)$ 同阶增长, 描述精确增长率。

小 o 记号 (Little-o) 定义: $f(n) = o(g(n))$ 表示对任意正常数 c , 存在 n_0 , 使得对所有 $n \geq n_0$ 有 $|f(n)| \leq c|g(n)|$ 。

含义: $f(n)$ 的增长严格慢于 $g(n)$ 。

例子: $2n = o(n^2)$ 。

ω 记号 (Little-omega) 定义: $f(n) = \omega(g(n))$ 表示对任意正常数 c , 存在 n_0 , 使得对所有 $n \geq n_0$ 有 $|f(n)| \geq c|g(n)|$ 。

含义: $f(n)$ 的增长严格快于 $g(n)$ 。

常见增长率排序 (从慢到快) 常数 $O(1) <$ 对数 $O(\log n) <$ 线性 $O(n) <$ 线性对数 $O(n \log n) <$ 多项式 $O(n^c)$ ($c > 1$) $<$ 指数 $O(c^n)$ ($c > 1$) $<$ 阶乘 $O(n!)$ 等。

注意: 比较增长率时需谨慎, 如 $2^{\log^2 n}$ 虽为指数形式, 但因指数是 $\log^2 n$, 故实际增长比多项式快但比真指数 (如 e^n) 慢。

1.2 字母表和语言 (Alphabets and Languages)

基本概念

- 字母表 Σ : 有限符号的集合, 如 $\Sigma = \{0, 1\}$ 。
- 字符串: 字母表中符号的无穷序列, 空串记为 ε 。
- 克林闭包 Σ^* : 由 Σ 中符号构成的所有字符串 (含空串) 的集合。
- 语言: Σ^* 的子集, 即一些字符串的集合。

语言运算

- 并、交、补: 与集合运算相同。
- 连接: $L_1 L_2 = \{xy \mid x \in L_1, y \in L_2\}$ 。
- 克林星号: $L^* = \{w_1 w_2 \dots w_k \mid k \geq 0, w_i \in L\}$, 即 L 中字符串的任意有限次连接。 $L^+ = LL^*$ 表示至少一次连接。(这是因为克林星号允许空串, 这里的 L^+ 正闭包强制一个因子来自 L , 不允许空串存在)

1.3 问题编码 (Encoding of Problems)

在计算理论中, 将问题实例编码为字符串以便计算模型 (如图灵机) 处理。编码应合理、简洁, 且能在多项式时间内解码。

二、作业题目详细讲解

题目 1

题目

证明

$$\frac{1}{2 \ln 2} + \frac{1}{3 \ln 3} + \dots + \frac{1}{n \ln n} = \ln \ln n + O(1)$$

和

$$\frac{1}{1^{1.1}} + \frac{1}{2^{1.1}} + \dots + \frac{1}{n^{1.1}} = O(1).$$

讲解

第一个等式证明 估计级数 $\sum_{k=2}^n \frac{1}{k \ln k}$ 。考虑函数 $f(x) = \frac{1}{x \ln x}$ ，在 $[2, \infty)$ 上为正且单调递减。对单调递减正函数，有

$$f(k+1) \leq \int_k^{k+1} f(x) dx \leq f(k) \quad (k \geq 2).$$

从 $k=2$ 累加到 n ，并将 $f(2)$ 单独留在上界中，得到

$$\int_2^{n+1} f(x) dx \leq \sum_{k=2}^n f(k) \leq f(2) + \int_2^n f(x) dx.$$

两侧都与 $\int_2^n f(x) dx$ 相比仅差有限常数，因此

$$\sum_{k=2}^n f(k) = \int_2^n f(x) dx + O(1).$$

计算积分：

$$\int \frac{1}{x \ln x} dx = \int \frac{1}{\ln x} d(\ln x) = \ln |\ln x| + C.$$

所以

$$\int_2^n \frac{1}{x \ln x} dx = \ln \ln n - \ln \ln 2.$$

因此

$$\sum_{k=2}^n \frac{1}{k \ln k} = \ln \ln n - \ln \ln 2 + O(1) = \ln \ln n + O(1),$$

其中常数 $-\ln \ln 2$ 被吸收到 $O(1)$ 中。

第二个等式证明 考虑级数 $\sum_{k=1}^n \frac{1}{k^{1.1}}$ 。由于指数 $1.1 > 1$ ，这是 p -级数且 $p > 1$ ，故无穷级数 $\sum_{k=1}^{\infty} \frac{1}{k^{1.1}}$ 收敛。收敛意味着部分和有上界，即存在常数 M 使得对所有 n 有 $\sum_{k=1}^n \frac{1}{k^{1.1}} \leq M$ ，所以 $\sum_{k=1}^n \frac{1}{k^{1.1}} = O(1)$ 。

更具体地，也可用积分估计上界：

$$\sum_{k=1}^n \frac{1}{k^{1.1}} \leq 1 + \int_1^n \frac{1}{x^{1.1}} dx = 1 + \left[\frac{x^{-0.1}}{-0.1} \right]_1^n = 1 + 10(1 - n^{-0.1}) \leq 11.$$

故确有界。

总结：第一个级数发散，发散速度为 $\ln \ln n$ ；第二个级数收敛，部分和有界。

题目 2

题目

将以下函数按渐近增长率排序（若 $f(n) = O(g(n))$ ，则 f 排在 g 前面）： $n^n - \sqrt{n} - \log \log n - 0.001n^7 - 5000n - e^n - 2^{\log^2 n} - 10^{10^{10}} - n! - 2^{2^n}$

讲解

需比较函数增长率，按从慢到快排序。分析各函数：

1. $10^{10^{10}}$ ：常数，增长率最低。
2. $\log \log n$ ：双对数，增长极慢，但比常数快。
3. $\sqrt{n} = n^{0.5}$ ：多项式（指数 0.5），比对数函数快（任何 n^c ($c > 0$) 均比 $\log n$ 快，而 $\log \log n$ 更慢）。
4. $5000n$ ：线性函数 n^1 ，比 $n^{0.5}$ 快。
5. $0.001n^7$ ：7 次多项式，比线性快。
6. $2^{\log^2 n}$ ：注意 $f(n) = 2^{\log^2 n}$ ，取对数得 $\log f(n) = \log^2 n$ 。多项式 n^c 的对数为 $c \log n$ ， $\log^2 n$ 增长比 $c \log n$ 快，故 $2^{\log^2 n}$ 比任何多项式快。但与指数函数 e^n 比较： $\log(e^n) = n$ ， n 增长比 $\log^2 n$ 快得多，所以 $2^{\log^2 n}$ 比 e^n 慢。因此它介于多项式与指数之间。
7. e^n ：指数函数，增长快。
8. $n!$ ：阶乘。由斯特林公式 $n! \sim \sqrt{2\pi n}(n/e)^n$ ，取对数 $\log(n!) \sim n \log n - n$ ，而 $\log(e^n) = n$ ， $n \log n$ 增长快于 n ，故 $n!$ 比 e^n 快。
9. n^n ：增长很快。
10. 2^{2^n} ：双指数函数，取对数得 $\log(2^{2^n}) = 2^n$ ，而 $\log(n^n) = n \log n$ ， 2^n 远快于 $n \log n$ ，故 2^{2^n} 比 n^n 快。

排序（从慢到快）：

$$10^{10^{10}} \prec \log \log n \prec \sqrt{n} \prec 5000n \prec 0.001n^7 \prec 2^{\log^2 n} \prec e^n \prec n! \prec n^n \prec 2^{2^n}$$

其中 $\prec$ 表示增长率小于。

关键点验证： $2^{\log^2 n}$ ：比多项式快（因为 $\log^2 n$ 比 $c \log n$ 快），但比指数慢（因为 $\log^2 n$ 比 n 慢）。 $n!$ 与 e^n ： $\log(n!) \sim n \log n > n = \log(e^n)$ ，故 $n!$ 更快。 n^n 与 2^{2^n} ： $\log(n^n) = n \log n$ ， $\log(2^{2^n}) = 2^n$ ， 2^n 更快。

题目 3

题目

令 $\Sigma = \{0, 1\}$ ，定义语言

$$L = \{w \in \{0,1\}^* : w \text{ 中 } 0 \text{ 和 } 1 \text{ 的数量不相等}\}.$$

证明 $L^* = \Sigma^*$ 。

讲解

需证集合相等： $L^* \subseteq \Sigma^*$ 且 $\Sigma^* \subseteq L^*$ 。

1. $L^* \subseteq \Sigma^*$ ：显然，因 L 中字符串均由 0 和 1 组成，故 L^* 中字符串也由 0 和 1 组成，属于 Σ^* 。
2. $\Sigma^* \subseteq L^*$ ：需证任意 $w \in \Sigma^*$ 可表示为 L 中若干字符串的连接。分情况：
 - 若 w 中 0 和 1 数量不相等，则 $w \in L \subseteq L^*$ 。
 - 若 w 中 0 和 1 数量相等，取最短前缀 x 使 x 中 0 和 1 数量差为 1，则剩余部分 y 的差为 -1 ，故 $x, y \in L$ ，从而 $w = xy \in L^*$ 。

综上， $\Sigma^* \subseteq L^*$ ，故 $L^* = \Sigma^*$ 。

复习小结：第一节课重点掌握渐近记号的含义与比较、积分估计级数和方法、语言运算及克林闭包的概念，并能进行简单的集合相等证明。

第二节课复习资料

一、本节课知识点讲解

2.1 有限自动机的定义

有限自动机 (FA) 是一种最简单的计算模型，它读取输入字符串，根据当前状态和输入符号决定下一个状态，最终根据结束状态是否在接受状态集合中来决定是否接受该字符串。

形式化定义（五元组） 一个有限自动机 M 是一个五元组 $(Q, \Sigma, \delta, q_0, F)$ ：

1. Q ：有限状态集合。
2. Σ ：输入字母表（有限符号集合）。
3. $\delta : Q \times \Sigma \rightarrow Q$ ：转移函数。给定当前状态和输入符号，输出下一个状态。
4. $q_0 \in Q$ ：初始状态。
5. $F \subseteq Q$ ：接受状态集合。

FA 如何接受字符串 设 $w = w_1 w_2 \cdots w_n$ ，其中每个 $w_i \in \Sigma$ 。 M 从 q_0 开始，依次读入每个符号，根据 δ 转移状态。读完所有符号后，若最终状态属于 F ，则 M 接受 w ；否则拒绝。

形式化： M 接受 w 当存在状态序列 $r_0, r_1, \dots, r_n$ 满足：

- $r_0 = q_0$
- $\delta(r_i, w_{i+1}) = r_{i+1}$ (对 $i = 0, \dots, n-1$)
- $r_n \in F$

正则语言 如果一个语言 L 被某个有限自动机接受，则称 L 是正则语言。记 $L(M)$ 为 M 接受的所有字符串的集合，即 M 识别的语言。

2.2 设计有限自动机的思路

设计 FA 的关键在于确定状态的含义。通常，状态用来记录读入部分字符串后所处的“情形”。例如：

- 记录当前读入的二进制数模某个数的余数。
- 记录是否已经出现了某个禁止子串。
- 记录最近读入的几个字符，以判断是否满足模式。

2.3 正则语言的封闭性

正则语言在多种运算下是封闭的，即对正则语言进行这些运算得到的语言仍然是正则的。封闭性证明通常通过构造新的 FA 来实现。

以下是常见的封闭运算（设 A, B 是正则语言）：

1. 补运算： $\overline{A} = \{w \in \Sigma^* \mid w \notin A\}$ 。
 - 证明：设 M 识别 A ，则交换 M 的接受与非接受状态得到 M' ， M' 识别 $\overline{A}$ 。
2. 并运算： $A \cup B$ 。
 - 证明：使用乘积自动机。设 M_1 识别 A ， M_2 识别 B ，构造 M 的状态为 M_1 和 M_2 状态的笛卡尔积（也就是一个状态变为“二元”的），转移模拟两者并行运行，接受状态定义为只要有一个自动机接受即可。
3. 交运算： $A \cap B$ 。
 - 证明：类似并运算，但接受状态定义为两个自动机同时接受。
4. 连接运算： $A \circ B = \{xy \mid x \in A, y \in B\}$ 。
 - 证明思路：非确定性有限自动机（NFA）更容易构造，参见第三讲知识点。
5. 星号运算： $A^* = \{x_1 x_2 \cdots x_k \mid k \geq 0, \text{每个 } x_i \in A\}$ 。
 - 证明思路：非确定性有限自动机（NFA）更容易构造，参见第三讲知识点。

注意：第二课中先介绍的是确定性有限自动机（DFA），但封闭性证明有时需要非确定性有限自动机（NFA）的概念，可见第三课。

二、作业题目详细讲解

题目 1

题目

设计一个有限自动机，接受以下语言

$$L = \left\{ w \in \{0, 1\}^* \mid w = w_1 w_2 \cdots w_n \text{ 使得 } \sum_{i=1}^n w_i 2^{n-i} \equiv 0 \pmod{5} \right\}.$$

换句话说，语言 L 是所有表示 5 的倍数的二进制字符串的集合，字符串以最高有效位优先（MSB）书写。绘制该有限自动机的状态图。

讲解

这个语言 L 中的字符串，如果解释为一个二进制数（最高位在前），这个数能被 5 整除。我们需要设计一个 DFA 来识别这样的字符串。

设计思路：

DFA 的状态应该记录当前已经读入的部分二进制数模 5 的余数。因为每读入一位，新的数值等于原来的数值左移一位（乘以 2）再加上新读入的位（0 或 1），然后模 5。所以，如果我们用状态 q_i 表示当前余数为 i ($i = 0, 1, 2, 3, 4$)，那么转移函数可以这样定义：

从状态 q_i 读入符号 b (0 或 1)，转移到状态 q_j ，其中 $j = (2 \times i + b) \bmod 5$ 。

初始状态应该是 q_0 ，因为还没有读入任何数字时，数值为 0，余数为 0。

接受状态也是 q_0 ，因为整个字符串读完时，我们希望余数为 0。

具体转移计算：

我们列出所有可能的状态和输入组合：

- 从 q_0 读入 0：新余数 $= (2 \times 0 + 0) \bmod 5 = 0$ ，所以转移到 q_0 。
- 从 q_0 读入 1：新余数 $= (2 \times 0 + 1) \bmod 5 = 1$ ，转移到 q_1 。
- 从 q_1 读入 0：新余数 $= (2 \times 1 + 0) \bmod 5 = 2$ ，转移到 q_2 。
- 从 q_1 读入 1：新余数 $= (2 \times 1 + 1) \bmod 5 = 3$ ，转移到 q_3 。
- 从 q_2 读入 0：新余数 $= (2 \times 2 + 0) \bmod 5 = 4$ ，转移到 q_4 。
- 从 q_2 读入 1：新余数 $= (2 \times 2 + 1) \bmod 5 = 5 \equiv 0$ ，转移到 q_0 。
- 从 q_3 读入 0：新余数 $= (2 \times 3 + 0) \bmod 5 = 6 \equiv 1$ ，转移到 q_1 。
- 从 q_3 读入 1：新余数 $= (2 \times 3 + 1) \bmod 5 = 7 \equiv 2$ ，转移到 q_2 。
- 从 q_4 读入 0：新余数 $= (2 \times 4 + 0) \bmod 5 = 8 \equiv 3$ ，转移到 q_3 。
- 从 q_4 读入 1：新余数 $= (2 \times 4 + 1) \bmod 5 = 9 \equiv 4$ ，转移到 q_4 。

状态图描述：

- 状态集合： $\{q_0, q_1, q_2, q_3, q_4\}$
- 输入字母表： $\{0, 1\}$
- 初始状态： q_0
- 接受状态： q_0
- 转移函数如上表。

我们可以绘制状态图：五个状态围成一个圆圈（顺序可为 q_0, q_1, q_2, q_3, q_4 ），每个状态根据上述转移有两条出边。注意 q_0 是初始状态（通常用箭头指向）和接受状态（通常用双圈表示）。

重要提醒：这个 DFA 也会接受空串 ε ，因为初始状态就是接受状态。在二进制数的表示中，空串通常不被视为合法的数字表示，但题目没有特别说明，按照题目定义，空串对应的数值为 0， $0 \bmod 5 = 0$ ，所以应该接受。如果要求非空字符串，则需要稍作修改，但题目没有此要求，所以这样设计是合理的。

题目 2

题目

设计一个有限自动机，识别以下语言

$$L = \{w \in \{0, 1\}^* : w \text{ 不包含子串 } 10\}.$$

绘制其状态图。

讲解

语言 L 包含所有不出现子串“10”的二进制字符串。也就是说，字符串中一旦出现了 1，后面就不能再出现 0，否则就会形成“10”。因此，合法的字符串必须是这样的形式：先有零个或多个 0，然后有零个或多个 1，即所有 0 都在所有 1 的前面。用正则表达式表示就是 0^*1^* 。

设计思路：

我们可以用三个状态来设计 DFA：

- 状态 q_0 ：表示目前还处于“0 的阶段”，即到目前为止读入的部分全部是 0（或者还没有读入任何字符）。在这个状态，我们还没有遇到 1。
- 状态 q_1 ：表示我们已经读到了至少一个 1，并且还没有出现 0 跟在 1 后面的情况。在这个状态，我们只希望后面继续读入 1，如果读到 0，就违反规则。
- 状态 q_2 ：表示已经出现了禁止子串“10”，即进入了“死亡状态”，一旦进入这个状态，无论读入什么符号，都将永远停留在这个状态，并且这个状态不是接受状态。

初始状态是 q_0 ，因为开始时还没有读入任何字符，属于 0 的阶段。

接受状态应该是 q_0 和 q_1 ，因为在这两个状态下，都没有出现禁止子串。

转移函数：

- 从 q_0 读入 0：仍然处于 0 的阶段，保持 q_0 。
- 从 q_0 读入 1：进入了 1 的阶段，转移到 q_1 。
- 从 q_1 读入 1：仍然在 1 的阶段，保持 q_1 。
- 从 q_1 读入 0：形成了“10”，禁止，转移到死亡状态 q_2 。
- 从 q_2 读入任何符号（0 或 1）：已经违规，永远留在 q_2 。

状态图描述：

- 状态集合： $\{q_0, q_1, q_2\}$
- 输入字母表： $\{0, 1\}$
- 初始状态： q_0
- 接受状态： $\{q_0, q_1\}$
- 转移函数：
 - $q_0 \xrightarrow{0} q_0$
 - $q_0 \xrightarrow{1} q_1$
 - $q_1 \xrightarrow{0} q_2$
 - $q_1 \xrightarrow{1} q_1$

$$\begin{aligned} & - q_2 \xrightarrow{0} q_2 \\ & - q_2 \xrightarrow{1} q_2 \end{aligned}$$

注意: q_2 不是接受状态, 且所有从 q_2 出发的转移都指向自己。

注意: 这个 DFA 也接受空串, 因为初始状态 q_0 是接受状态。语言 0^*1^* 确实包含空串。

题目 3

题目

使用大 O 符号的定义证明以下内容:

- 1) 如果 $f_1 = O(g_1)$ 且 $f_2 = O(g_2)$, 则 $f_1 f_2 = O(g_1 g_2)$ 。
- 2) 如果 $f_1 = O(g)$ 且 $f_2 = O(g)$, 则 $f_1 + f_2 = O(g)$ 。

讲解

我们需要回顾大 O 符号的严格定义:

$f(n) = O(g(n))$ 意味着存在正常数 C 和 N , 使得对于所有 $n > N$, 都有 $|f(n)| \leq C|g(n)|$ 。

证明第 1 小题 已知: $f_1 = O(g_1)$, 即存在常数 $C_1 > 0$ 和 N_1 , 使得对所有 $n > N_1$, 有 $|f_1(n)| \leq C_1|g_1(n)|$ 。

已知: $f_2 = O(g_2)$, 即存在常数 $C_2 > 0$ 和 N_2 , 使得对所有 $n > N_2$, 有 $|f_2(n)| \leq C_2|g_2(n)|$ 。

我们取 $N = \max(N_1, N_2)$, 那么当 $n > N$ 时, 两个已知不等式都成立。

将两个不等式相乘:

$$|f_1(n)| \cdot |f_2(n)| \leq C_1|g_1(n)| \cdot C_2|g_2(n)| = (C_1 C_2)|g_1(n)g_2(n)|。$$

由于 $|f_1(n)f_2(n)| = |f_1(n)| \cdot |f_2(n)|$, 故有

$$|f_1(n)f_2(n)| \leq (C_1 C_2)|g_1(n)g_2(n)|。$$

因此, 取 $C = C_1 C_2$, 则对于所有 $n > N$, 不等式成立。所以 $f_1 f_2 = O(g_1 g_2)$ 。

证明第 2 小题 已知: $f_1 = O(g)$, 即存在常数 $C_1 > 0$ 和 N_1 , 使得对所有 $n > N_1$, 有 $|f_1(n)| \leq C_1|g(n)|$ 。

已知: $f_2 = O(g)$, 即存在常数 $C_2 > 0$ 和 N_2 , 使得对所有 $n > N_2$, 有 $|f_2(n)| \leq C_2|g(n)|$ 。

目标: 证明 $f_1 + f_2 = O(g)$, 即要找到常数 C 和 N , 使得对所有 $n > N$, 有 $|f_1(n) + f_2(n)| \leq C|g(n)|$ 。

我们取 $N = \max(N_1, N_2)$, 那么当 $n > N$ 时, 两个已知不等式都成立。

利用三角不等式:

$$|f_1(n) + f_2(n)| \leq |f_1(n)| + |f_2(n)| \leq C_1|g(n)| + C_2|g(n)| = (C_1 + C_2)|g(n)|。$$

因此, 取 $C = C_1 + C_2$, 则对于所有 $n > N$, 有 $|f_1(n) + f_2(n)| \leq C|g(n)|$ 。所以 $f_1 + f_2 = O(g)$ 。

复习小结: 第二节课重点掌握有限自动机的形式化定义、设计方法 (特别是基于余数或禁止模式的状态设计), 以及正则语言封闭性的基本证明思路 (补、并)。同时, 大 O 记号的性质证明需严格按照定义进行构造。

第三节课复习资料

一、本节课知识点讲解

3.1 非确定性有限自动机 (NFA)

NFA 的定义 NFA 与 DFA 类似，但转移函数可以是非确定性的：即从一个状态读入一个符号，可以转移到多个可能的状态（包括零个）。此外，NFA 允许 ε 转移，即不读入任何符号就改变状态。

形式化定义：一个 NFA 是一个五元组 $(Q, \Sigma, \delta, q_0, F)$ ，其中：

- Q ：有限状态集合。
- Σ ：输入字母表。
- $\delta: Q \times (\Sigma \cup \{\varepsilon\}) \rightarrow \mathcal{P}(Q)$ ，其中 $\mathcal{P}(Q)$ 是 Q 的幂集（即 Q 中所有状态都可能存在， 2^Q 种）。
- $q_0 \in Q$ ：初始状态。
- $F \subseteq Q$ ：接受状态集合。

NFA 接受一个字符串 w ，如果存在一条从初始状态到某个接受状态的路径，路径上的转移标记依次与 w 的符号匹配（允许 ε 转移，不消耗输入符号）。

NFA 与 DFA 的等价性 对于任意 NFA，存在一个 DFA 接受相同的语言（反之亦然）。DFA 本身显然就是一种 NFA，而反过来证明可使用子集构造法：将 NFA 的状态集合的子集作为 DFA 的状态（关键！我们构造的 DFA，其状态应是已知 NFA 的某些状态的集合），以此模拟 NFA 所有可能的并行状态。

子集构造法：给定 NFA $N = (Q, \Sigma, \delta, q_0, F)$ ，构造 DFA $M = (Q', \Sigma, \delta', q'_0, F')$ ：

- $Q' = \mathcal{P}(Q)$ 。
- $q'_0 = E(\{q_0\})$ ，其中 $E(R)$ 表示从集合 R 中的状态通过 ε 转移所能到达的状态集合（ ε 闭包），相当于把 NFA 中可以用来出发的状态“打包”了。
- 对于 $S \subseteq Q$ 和 $a \in \Sigma$ ，定义 $\delta'(S, a) = E\left(\bigcup_{r \in S} \delta(r, a)\right)$ 。这样，我们就能得到我们构造的 DFA 中的一个“打包”遇到一个输入字母时，应去往哪一个“打包”。
- $F' = \{S \subseteq Q : S \cap F \neq \emptyset\}$ 。因为我们的最终目标是让构造的 DFA 和原始的 NFA 等价，因此凡是包含 F 中状态的的状态集合都应作为我们构造的 DFA 的接收状态。

正则语言的封闭性（连接与星号） 利用 NFA 可以方便地证明正则语言在连接和星号运算下封闭。

- **连接**：若 L_1 和 L_2 是正则语言，则 $L_1 \circ L_2 = \{xy : x \in L_1, y \in L_2\}$ 也是正则的。证明：设 N_1 接受 L_1 ， N_2 接受 L_2 。构造 NFA N ：将 N_1 的每个接受状态通过 ε 转移连接到 N_2 的初始状态。 N 的初始状态为 N_1 的初始状态，接受状态为 N_2 的接受状态。
- **星号**：若 L 是正则语言，则 $L^* = \{x_1 x_2 \cdots x_k : k \geq 0, x_i \in L\}$ 也是正则的。证明：设 N_1 接受 L 。构造 NFA N ：新增一个初始状态 q_0 （也是接受状态），从 q_0 通过 ε 转移到 N_1 的初始状态；同时从 N_1 的接受状态通过 ε 转移回到 N_1 的初始状态（这样就允许了重复）。

3.2 正则表达式

正则表达式是一种描述正则语言的形式化方法。递归定义如下：

1. 基础:

- a ($a \in \Sigma$) 是正则表达式, 表示语言 $\{a\}$ 。
- ε 是正则表达式, 表示语言 $\{\varepsilon\}$ 。(注意, 这里的 $\{\varepsilon\}$ 中只包含一个元素, 即空串, 不能和空集混淆)
- $\emptyset$ 是正则表达式, 表示空语言。

2. 归纳:

- 如果 r 和 s 是正则表达式, 表示语言 $L(r)$ 和 $L(s)$, 则:
 - $(r \cup s)$ 是正则表达式, 表示 $L(r) \cup L(s)$ 。
 - $(r \circ s)$ 是正则表达式, 表示 $L(r) \circ L(s)$ 。
 - (r^*) 是正则表达式, 表示 $L(r)^*$ 。

通常, 运算符优先级为: 星号 > 连接 > 并集。括号可省略。

正则表达式与有限自动机等价: 每个正则表达式对应一个正则语言, 反之亦然。证明可通过将正则表达式转换为 NFA (递归构造), 或反之 (状态消去法)。

二、作业题目详细讲解

题目 1

题目

定义一个反向读取 DFA 为从右向左处理输入字符串的确定性有限自动机。证明一个语言是正则的当且仅当它可以被某个反向读取 DFA 接受。

讲解

我们需要证明两个方向。

方向 1: 如果语言 L 是正则的, 那么存在一个反向读取 DFA 接受 L 。

证明: 因为 L 是正则语言, 所以存在一个普通的 DFA $M = (Q, \Sigma, \delta, q_0, F)$ 接受 L 。我们构造一个反向读取 DFA M' 如下:

- M' 的状态集与 M 相同: $Q' = Q$ 。
- M' 的输入字母表与 M 相同: $\Sigma' = \Sigma$ 。
- M' 的转移函数 δ' 定义为: 对于任意 $q \in Q$ 和 $a \in \Sigma$, 令 $\delta'(q, a) = p$ 当且仅当在 M 中有 $\delta(p, a) = q$ 。也就是说, M' 的转移是 M 转移的反向。
- M' 的初始状态设为 M 的任意一个接受状态。

上述做法在转移函数构造和初始状态构造上, 是否存在不严谨的地方呢? 实际是有的: 1) 把 $\delta'(q, a)$ 定义为“所有前驱”后, 它可能不是单值, 所得自动机其实是 NFA 而非 DFA; 2) 初始状态不能随便选取一个接受状态, 而应该同时考虑所有接受状态的反向可达性, 否则会漏掉路径。

可以考虑: 先构造一个 NFA N , 使得 N 接受 L 的反转语言 $L^R = \{w^R : w \in L\}$, 其中 w^R 是 w 的翻转。具体构造:

- 先在 M 的基础上新增一个初始状态 s ，从 s 通过 ε 转移到每个 $f \in F$ ；设 $N = (Q \cup \{s\}, \Sigma, \delta_N, s, \{q_0\})$ 。将这个 s 作为初始状态，就巧妙地解决了上述的“所有接受状态的反向可达性”问题，因为 s 可以到达这些状态，现在就等价于 M 的最终状态了。
- 定义反向转移：对任意 $q \in Q, a \in \Sigma$ ，令 $\delta_N(q, a) = \{p \in Q : \delta(p, a) = q\}$ （这样就解决了上述“可能不是单值”问题）；其他未定义的转移为空集。
- 最后， $\{q_0\}$ 可构造为唯一接受状态。

这个 NFA N 接受的语言正好是 L^R 。因为 L 是正则的，所以 L^R 也是正则的（正则语言在反转下封闭，已证结论）。然后，将 NFA N 通过子集构造法转化为 DFA D （前面已证），这个 DFA D 接受 L^R 。最后，说明让其反向读取即可。因此，存在反向读取 DFA 接受 L 。

方向 2：如果存在一个反向读取 DFA 接受语言 L ，那么 L 是正则的。

证明：设 M' 是一个反向读取 DFA，接受 L 。定义 M' 正向读（从左向右）时接受的语言为 L' 。由于 M' 是 DFA，所以 L' 是正则语言。但注意 M' 反向读时接受 L ，正向读时实际上接受的是 L 的反转 L^R 。因此 $L^R = L'$ 是正则的。由于正则语言在反转下封闭，所以 $L = (L^R)^R$ 也是正则的。因此直接得证。

综上，语言 L 是正则的当且仅当它可以被某个反向读取 DFA 接受。

题目 2

题目

给出语言 $L = \{w \in \{0,1\}^* : w \text{ 中 } 1 \text{ 的个数是偶数, 且 } 0 \text{ 的个数也是偶数}\}$ 的正则表达式。

讲解

我们想要所有包含偶数个 0 和偶数个 1 的二进制字符串。考虑用模 2 的状态来思考。但直接写正则表达式，可以注意到：任何这样的字符串，都可以看成由以下四种“片段”组成，这些片段在添加时不会改变 0 和 1 的奇偶性（即添加后，0 和 1 的个数的奇偶性保持不变）：

- “00”：添加两个 0，不改变奇偶性。
- “11”：添加两个 1，不改变奇偶性。
- “01”后跟“10”：即“0110”，整体添加了两个 0 和两个 1，奇偶性不变。类似地，“10”后跟“01”也是。

因此，最简洁的表达式是：

$$(00 \cup 11 \cup (01 \cup 10)(00 \cup 11)^*(01 \cup 10))^*$$

解释：

- $00 \cup 11$ ：添加两个相同字符，奇偶性不变。
- $(01 \cup 10)(00 \cup 11)^*(01 \cup 10)$ ：先添加一个“01”或“10”，这会翻转 0 和 1 的奇偶性（使 0 和 1 都变成奇数个），然后添加任意个“00”或“11”（不改变奇偶性），最后再添加一个“01”或“10”，再次翻转奇偶性，回到偶数。整体效果是奇偶性不变。

整个表达式外面加星号，表示重复任意多次（包括零次），所以最终得到的就是所有偶数个 0 和偶数个 1 的字符串。

注意：这个表达式也包含空串，因为星号允许零次重复，而空串中 0 和 1 的个数都是 0（偶数），符合要求。

题目 3

题目

构造与以下正则表达式对应的 NFA: 1. $0^+ \cup (01)^*$ 2. $(0 \cup 1^+)0^*1^+$

讲解

我们分别构造两个正则表达式的 NFA。NFA 允许 ϵ 转移，并且每个部分可以独立构造然后组合。

第一问: $0^+ \cup (01)^*$ 这个语言表示: 要么是一个或多个 0 的序列, 要么是零个或多个 “01” 的序列。

我们构造 NFA 如下:

- 状态集合: $\{q_0, q_1, q_2, q_3, q_4, q_5\}$, 其中 q_0 是初始状态。
- 输入字母表: $\{0, 1\}$ 。
- 接受状态: q_2 (对于 0^+ 部分) 以及 q_3 和 q_5 (对于 $(01)^*$ 部分)。

转移函数:

1. 从 q_0 通过 ϵ 转移到 q_1 (进入 0^+ 分支) 和通过 ϵ 转移到 q_3 (进入 $(01)^*$ 分支)。
2. 0^+ 部分:
 - $q_1 \xrightarrow{0} q_2$ 。
 - $q_2 \xrightarrow{0} q_2$ (自环, 允许更多 0)。
3. $(01)^*$ 部分:
 - q_3 是接受状态 (因为 $(01)^*$ 允许空串)。
 - $q_3 \xrightarrow{0} q_4$ 。
 - $q_4 \xrightarrow{1} q_5$ 。
 - q_5 是接受状态。
 - $q_5 \xrightarrow{0} q_4$ (以便继续下一个 “01”)。

这样, 对于 $(01)^*$, 空串可以直接在 q_3 接受; 非空的 “01” 序列会经过 $q_3 \rightarrow q_4 \rightarrow q_5$, 并且从 q_5 读 0 可以回到 q_4 继续循环。

第二问: $(0 \cup 1^+)0^*1^+$ 这个语言表示: 首先是一个 0 或者一个或多个 1, 然后是零个或多个 0, 最后是一个或多个 1。

我们构造 NFA 如下:

- 状态集合: $\{q_1, q_2, q_3, q_4, q_5\}$, 其中 q_1 是初始状态。
- 输入字母表: $\{0, 1\}$ 。
- 接受状态: q_5 。

转移函数:

1. 第一部分 $(0 \cup 1^+)$:
 - $q_1 \xrightarrow{0} q_2$ (选择 0)。
 - $q_1 \xrightarrow{1} q_3$ (选择 1^+)。
 - $q_3 \xrightarrow{1} q_3$ (自环, 允许更多 1)。

2. 连接第二部分 0^* :

- 从 q_2 和 q_3 通过 ε 转移到 q_4 。
- $q_4 \xrightarrow{0} q_4$ (自环, 允许任意多个 0)。

3. 连接第三部分 1^+ :

- $q_4 \xrightarrow{1} q_5$ (第一个 1)。
- $q_5 \xrightarrow{1} q_5$ (自环, 允许更多 1)。

这样, NFA 从 q_1 开始, 根据第一个字符选择分支, 然后通过 ε 转移到 q_4 , 在 q_4 可以读任意多个 0, 最后必须读至少一个 1 到达 q_5 并可以继续读更多 1。 q_5 是接受状态。

注意: 这个构造中, 从 q_2 和 q_3 到 q_4 的 ε 转移是必要的, 因为 0^* 部分允许零个 0, 所以不能直接合并状态。而 q_4 到 q_5 的转移是读 1, 没有使用 ε 转移, 这样更简洁。

复习小结: 第三节课重点掌握 NFA 的定义及其与 DFA 的等价性 (子集构造法)、正则表达式的递归定义、以及利用 NFA 证明正则语言在连接和星号运算下的封闭性。同时, 要能够进行正则表达式与有限自动机之间的转换 (构造 NFA)。作业题目涉及反向 DFA 的等价性证明、构造特定语言的正则表达式、以及从正则表达式构造 NFA, 需要理解并熟练运用相关概念和构造技巧。

第四节课复习资料

一、本节课知识点讲解

4.1 泵引理 (Pumping Lemma)

泵引理是用于证明一个语言不是正则的有力工具。其基本思想是: 如果一个语言是正则的, 那么它能被一个 DFA 识别。DFA 只有有限个状态 (设其状态数为 p), 那么任何长度大于 p 的字符串在运行过程中必然会重复经过某个状态。利用这个重复, 我们可以将字符串的某部分 “泵入” 任意多次, 得到的字符串仍然应该被接受。

泵引理 (正则语言的泵引理): 设 L 是一个正则语言, 则存在一个正整数 p (称为泵长度) 使得对于 L 中任意长度至少为 p 的字符串 s , 都可以将 s 写成 $s = xyz$ 的形式, 满足以下三个条件:

1. 对任意 $i \geq 0$, $xy^iz \in L$;
2. $|y| > 0$;
3. $|xy| \leq p$ 。

需要注意的是, 泵引理是正则语言的必要条件, 而不是充分条件。也就是说, 如果一个语言满足泵引理的条件, 我们不能说它一定是正则的; 但如果不满足, 则它一定不是正则的。

证明思路: 设 M 是一个有 p 个状态的 DFA, 且 $L(M) = L$ 。对于任意长度 $n \geq p$ 的字符串 $s = s_1s_2 \cdots s_n \in L$, 考虑 M 在输入 s 上的状态序列 $r_0, r_1, \dots, r_n$, 其中 r_0 是初始状态, 且 r_n 是接受状态。因为状态只有 p 个, 而序列有 $n+1 > p$ 个状态, 根据鸽巢原理, 必有两个状态相同。设 $r_j = r_l$ 是第一次出现重复的状态, 且 $0 \leq j < l \leq p$ 。那么, 取 $x = s_1 \cdots s_j$, $y = s_{j+1} \cdots s_l$, $z = s_{l+1} \cdots s_n$, 则 $|y| > 0$, $|xy| \leq p$ 。由于 $r_j = r_l$, 所以从 r_j 读入 y 会回到 r_j , 因此对任意 $i \geq 0$, xy^iz 都会到达同一个接受状态 r_n , 即 $xy^iz \in L$ 。

4.2 泵引理的应用

使用泵引理证明语言 L 不是正则的一般步骤：

1. 假设 L 是正则的，则存在泵长度 p 。
2. 选择一个合适的字符串 $s \in L$ ，且 $|s| \geq p$ 。选择 s 时需要技巧，通常要使泵引理的条件能导致矛盾。
3. 对于所有满足泵引理条件的 s 的划分 $s = xyz$ （即 $|xy| \leq p$ ， $|y| > 0$ ），都要能导出矛盾，即 xy^iz 不在 L 之中。因此，需要根据 p 来构造 s ，使得无论 y 怎么取（只要满足条件），通过泵入（例如取 $i = 0$ 或 $i = 2$ 等）都会得到一个不在 L 中的字符串。
4. 因此假设错误， L 不是正则的。

常见的非正则语言例子：

- $\{0^n 1^n : n \geq 0\}$;
- $\{ww : w \in \{0, 1\}^*\}$;
- $\{0^p : p \text{ 是素数}\}$ 。

4.3 从 NFA 到正则表达式的转换

另一种重要的技术是将有限自动机转换为等价的正则表达式。这可以通过状态消除法实现，也可以使用广义非确定性有限自动机（GNFA）。

GNFA（广义非确定性有限自动机）：GNFA 是一种特殊的 NFA，其转移边上的标签可以是任意正则表达式，而不仅仅是单个符号或 ε 。GNFA 有一个唯一的开始状态（没有入边）和一个唯一的接受状态（没有出边）。其他状态之间都有边（如果没有实际转移，则标签为 $\emptyset$ ）。

转换步骤：

1. 给定一个 NFA，首先添加一个新的开始状态（用 ε 边连接到原开始状态）和一个新的接受状态（从所有原接受状态用 ε 边连接到此状态）。这样确保开始状态没有入边，接受状态没有出边。
2. 逐步消除中间状态。每次消除一个状态 q ，对于每一对状态 q_i （指向 q ）和 q_j （从 q 指出），我们需要更新从 q_i 到 q_j 的边上的正则表达式，以反映经过 q 的所有路径。具体地，假设从 q_i 到 q 的边标签为 R_1 ，从 q 到自身的自环标签为 R_2 （如果没有自环，则 $R_2 = \emptyset$ ），从 q 到 q_j 的边标签为 R_3 。那么消除状态 q 后，从 q_i 到 q_j 的路径新增了通过 q 的路径，其正则表达式为 $R_1 R_2^* R_3$ 。将这个表达式与原来从 q_i 到 q_j 的边标签（设为 R_4 ）合并，得到新的标签为 $R_4 \cup R_1 R_2^* R_3$ 。
3. 重复消除中间状态，直到只剩下开始状态和接受状态。此时，从开始状态到接受状态的边上的正则表达式就是所求。

注意：在消除状态时，需要更新所有受影响的边。可能还需要处理自环。

状态消除法：与 GNFA 方法类似，但更直观。直接在原 NFA 上操作，每次选择一个状态消除，并重新连接其他状态。

二、作业题目详细讲解

题目 1

题目

设 $L = \{0^p : p \text{ 是素数}\}$ 。证明 L 不是正则的。

讲解

我们使用泵引理。假设 L 是正则的，那么存在泵长度 p (正整数)。我们要选择一个合适的字符串 $s \in L$ ，且 $|s| \geq p$ ，然后说明对于任何满足泵引理条件的划分，都能导出矛盾。

注意：泵引理是说存在一个 p ，使得对于任意长度 $\geq p$ 的字符串 s ，存在一种划分 $s = xyz$ 满足条件。所以当我们假设 L 正则时，我们并不知道 p 具体是多少，但我们可以根据 p 来构造 s 。通常我们选择 $s = 0^q$ ，其中 q 是一个大于等于 p 的素数。这样的素数总是存在的（比如取大于 p 的某个素数）。

设 $s = 0^q$ ，其中 $q \geq p$ 且 q 是素数。根据泵引理， s 可以写成 $s = xyz$ ，满足：

1. $|xy| \leq p$;
2. $|y| > 0$;
3. 对于所有 $i \geq 0$ ， $xy^iz \in L$ 。

由于 $|xy| \leq p$ 且 s 全部由 0 组成，所以 y 也全部由 0 组成，设 $|y| = k$ ，则 $1 \leq k \leq p$ 。那么 xy^iz 就是 $0^{q+(i-1)k}$ ，因为 x 和 z 也是 0。

根据条件 3，对于任意 $i \geq 0$ ， $xy^iz \in L$ ，即 $0^{q+(i-1)k}$ 的指数必须是素数。特别地，取 $i = q + 1$ ，则 $xy^{q+1}z = 0^{q+qk} = 0^{q(1+k)}$ 。因为 $k \geq 1$ ，所以 $1 + k \geq 2$ ，且 $q \geq 2$ ，因此 $q(1+k)$ 是两个大于 1 的整数的乘积，是合数。所以 $0^{q(1+k)}$ 不在 L 中，矛盾。因此假设错误， L 不是正则的。

注意：这里我们选择 $i = q + 1$ 是为了让指数变得明显是合数。实际上也可以选择其他 i ，但需要确保指数不是素数。例如，取 $i = 0$ ，得到 0^{q-k} ，但 $q - k$ 可能是素数也可能不是，所以不一定矛盾。因此选择 i 使得指数明显是合数很重要。

题目 2

题目

设 $L = \{www : w \in \{0,1\}^*\}$ 。证明 L 不是正则的。

讲解

同样使用泵引理。假设 L 是正则的，存在泵长度 p 。

我们选择字符串 $s = 0^p 1^p 0^p 1^p 0^p 1^p$ 。这个字符串可以看成三个相同的部分 www ，其中 $w = 0^p 1^p$ 。显然 $s \in L$ ，且 $|s| = 6p \geq p$ 。

根据泵引理， s 可以写成 $s = xyz$ ，满足 $|xy| \leq p$ ， $|y| > 0$ ，且对于所有 $i \geq 0$ ， $xy^iz \in L$ 。

由于 $|xy| \leq p$ ，且 s 的前 p 个字符都是 0，所以 y 一定完全由 0 组成，设 $|y| = k$ ， $1 \leq k \leq p$ 。

现在考虑 $i = 0$ ，即泵出 y ，得到字符串 $xz = 0^{p-k} 1^p 0^p 1^p 0^p 1^p$ 。如果 $xz \in L$ ，那么它必须能写成三个完全相同的子串连接的形式。观察 xz ，它由三段组成：第一段是 $0^{p-k} 1^p$ ，第二段是 $0^p 1^p$ ，第三段是 $0^p 1^p$ 。由

于 $k > 0$ ，第一段中 0 的个数为 $p - k$ ，小于第二段和第三段中 0 的个数 p ，因此这三个子串不可能完全相同。所以 $xz \notin L$ ，与泵引理矛盾。因此 L 不是正则的。

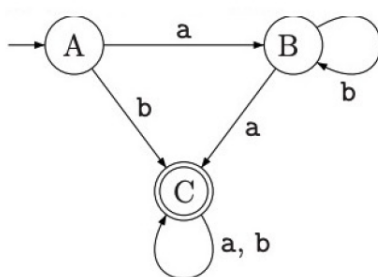
注意：这里选择 $i = 0$ 是因为可以破坏三个部分的一致性。当然也可以选择其他 i ，比如 $i = 2$ ，得到 xy^2z ，那么第一个子串中 0 的个数会增加，但第二个子串不变，同样会破坏一致性。但选择 $i = 0$ 是最简单的。

题目 3

题目

应用课堂上的步骤将给定的 NFA 转换为正则表达式。

补充描述：状态转移图如下：



- 状态 A （初始状态）、 B 、 C （接受状态，双圈）。
- 转移： $A \rightarrow B$ 上有标签 a ； $A \rightarrow C$ 上有标签 b ； $B \rightarrow C$ 上有标签 a ； $B \rightarrow B$ 上有标签 b （自环）； $C \rightarrow C$ 上有标签 a, b （自环）。

讲解

我们使用状态消除法。目标是将 NFA 转换为等价的正则表达式。

步骤 1：将 NFA 转化为 GNFA 形式

我们添加新的开始状态 S 和新的接受状态 F ，并添加 ε 转移：

- 从 S 到 A ： ε
- 从 C 到 F ： ε （因为 C 是原接受状态）

现在，我们有一个 GNFA，状态集合为 $\{S, A, B, C, F\}$ 。

步骤 2：消除状态 A

状态 A 有入边： $S \rightarrow A(\varepsilon)$ ；出边： $A \rightarrow B(a)$ ， $A \rightarrow C(b)$ 。

消除 A 时，我们需要考虑经过 A 的路径：

- $S \rightarrow A \rightarrow B$ ： $\varepsilon a = a$
- $S \rightarrow A \rightarrow C$ ： $\varepsilon b = b$

将这两条路径加到已有的边上（ S 到 B 原来没有边，所以直接添加； S 到 C 原来没有边，直接添加）。所以：

- 添加 $S \rightarrow B$ ： a
- 添加 $S \rightarrow C$ ： b

同时， S 到其他状态的边不变。然后移除状态 A 。

此时 GNFA 状态为 $\{S, B, C, F\}$ ，转移如下：

- $S \rightarrow B: a$
- $S \rightarrow C: b$
- $B \rightarrow B: b$ (自环)
- $B \rightarrow C: a$
- $C \rightarrow C: a \cup b$ (自环)
- $C \rightarrow F: \varepsilon$

步骤 3：消除状态 B

状态 B 有入边： $S \rightarrow B(a)$ ， $B \rightarrow B(b)$ (自环)；出边： $B \rightarrow C(a)$ 。

消除 B 时，要注意 B 有自环，从 S 到 C 的路径可以通过 B 任意次：

- 从 S 到 C 经过 B ： $S \rightarrow B(a)$ ，然后 B 的自环 b^* ，然后 $B \rightarrow C(a)$ ，所以路径为 ab^*a 。
- 另外，从 S 到 C 原来有直接边 b (来自 $S \rightarrow C$)，所以合并后 $S \rightarrow C$ 的标签更新为 $b \cup ab^*a$ 。

消除 B 后，状态变为 $\{S, C, F\}$ ，转移如下：

- $S \rightarrow C: b \cup ab^*a$
- $C \rightarrow C: a \cup b$ (自环)
- $C \rightarrow F: \varepsilon$

步骤 4：消除状态 C

状态 C 有入边： $S \rightarrow C(b \cup ab^*a)$ ， $C \rightarrow C(a \cup b)$ (自环)；出边： $C \rightarrow F(\varepsilon)$ 。

消除 C 时，从 S 到 F 的路径： $S \rightarrow C(b \cup ab^*a)$ ，然后 C 的自环 $(a \cup b)^*$ ，然后 $C \rightarrow F(\varepsilon)$ 。所以 S 到 F 的路径标签为 $(b \cup ab^*a)(a \cup b)^*$ 。

因此，所求正则表达式为 $(b \cup ab^*a)(a \cup b)^*$ 。

复习小结：第四节课重点掌握泵引理的内容、证明思路和应用，以及如何使用状态消除法（或 GNFA）将 NFA 转换为正则表达式。作业题目分别练习了泵引理证明非正则语言和 NFA 到正则表达式的转换。

第五节课复习资料

一、本节课知识点讲解

5.1 图灵机的定义与基本操作

图灵机是计算理论中最核心的计算模型，它由一个无限长的纸带、一个读写头和一套控制规则（状态转移函数）组成。图灵机提供了算法执行的形式化模型。

形式化定义（七元组） 一个（确定型）图灵机 M 是一个七元组 $(Q, \Sigma, \Gamma, \delta, q_0, q_{accept}, q_{reject})$ ，其中：

1. Q : 有限的状态集合。
2. Σ : 输入字母表 (不包含空白符 $\sqcup$ 或 $\sqcup$, 注意这里的 $\sqcup$ 是空白符, 字符串之外的所有格子都默认填充它)。
3. Γ : 纸带字母表, 满足 $\Sigma \subseteq \Gamma$ 且包含空白符 (通常记作 $\sqcup, B$ 或 $_$)。
4. δ : 转移函数, $\delta: Q \times \Gamma \rightarrow Q \times \Gamma \times \{L, R\}$ 。
 - 输入: 当前状态 q 和读写头下方符号 a 。
 - 输出: 新状态 q' 、要写入的符号 b 以及读写头的移动方向 (L 左移或 R 右移)。
5. $q_0 \in Q$: 初始状态。
6. $q_{accept} \in Q$: 接受状态 (唯一, 进入此状态则停机并接受输入)。
7. $q_{reject} \in Q$: 拒绝状态 (唯一, 进入此状态则停机并拒绝输入)。

图灵机如何运行

1. 开始时, 输入字符串写在纸带上, 其余格子为空白符。读写头指向输入最左端的符号。机器处于初始状态 q_0 。
2. 每一步, 机器根据当前状态 q 和读写头下符号 a , 查找转移函数 $\delta(q, a) = (q', b, D)$ 。
 - 将当前格子的符号改写为 b 。
 - 将状态变为 q' 。
 - 根据 D 将读写头向左 (L) 或向右 (R) 移动一格。
3. 当机器进入 q_{accept} 或 q_{reject} 状态时, 立即停机, 并相应输出“接受”或“拒绝”。
4. 如果转移函数 δ 对当前 (q, a) 无定义, 机器也会停机 (通常视为拒绝)。

图灵机的基本算法模式

- 扫描与计数: 通过左右移动读写头来遍历字符串, 用不同的符号进行标记, 以计数或检查模式。
- 记忆与比较: 通过状态或纸带上的特殊标记来“记住”之前读取的符号, 以便与后续符号进行比较 (例如判断回文)。
- 迭代与循环: 通过设计状态间的循环转移, 实现对某一段纸带的重复处理。

时间复杂度 (初步) 对于判定语言 $L \subseteq \{0, 1\}^*$ 的图灵机 M , 我们说 M 在时间 $T(n)$ 内判定 L , 如果对于每个长度为 n 的输入 x :

1. M 在至多 $T(|x|)$ 步内必然停机 (到达 q_{accept} 或 q_{reject})。
2. 若 $x \in L$, 则 M 接受 x 。
3. 若 $x \notin L$, 则 M 拒绝 x 。

注意: 这里的 $L \subseteq \{0, 1\}^*$ 并不是特例, 而是因为计算机科学中任何对象 (整数、图、程序代码、矩阵等) 都可以通过编码映射为一段二进制字符串。此外, 这个概念是后续讨论计算复杂性 (P、NP 类) 的基础。

二、作业题目详细讲解

题目 1

题目

输入是两个二进制数 x 和 y （最高位在左边），由符号 $\#$ 分隔，例如 $100\#100$ 。假设没有前导零，除非数字本身是 0。设计一个图灵机，如果 x 和 y 相等，则接受。

讲解

我们的目标是逐位比较 x 和 y 的每一位。由于输入格式固定为 $x\#y$ ，且没有前导零，我们可以从左到右依次处理。

核心思路：

1. 逐位配对与标记：从 x 的最左端开始，读取一位（0 或 1），将其标记为一个特殊符号（例如 X ），然后向右移动找到 $\#$ ，再继续向右找到 y 的对应位（第一个尚未被标记的位）。检查该位是否与 x 刚才读取的位相同。
2. 匹配处理：如果相同，则将 y 的该位标记为另一个特殊符号（例如 Y ），然后向左移动回到 x 部分，找到下一个未被标记的位，重复此过程。
3. 长度与结束检查：在比较过程中，如果出现位数不匹配（例如 x 还有位但 y 已到末尾，或符号不匹配），则立即拒绝。当 x 的所有位都被标记后，需要检查 y 是否也恰好没有剩余位了。如果是，则接受。

状态设计与转移逻辑：

- **start**: 初始状态。查看第一个符号。
 - 如果是 $\#$ ，说明 x 为空，则直接跳到检查 y 是否也为空的状态 (**check_y_end**)。
 - 如果是 0 或 1，将其改写为 X （标记为已处理），并分别进入状态 **check0** 或 **check1**，开始寻找 y 的对应位。
 - 如果是空白 ($\square$)，说明输入完全为空（ x 和 y 都是空串），直接接受。
- **check0 / check1**: 向右移动，跳过已经处理过的 0、1（理论上这里不会出现 X ），寻找分隔符 $\#$ 。
- **verify0 / verify1**: 在找到 $\#$ 后进入此状态，继续向右跳过 Y （ y 中已匹配的位），寻找 y 的第一个未匹配位。它期望找到一个与 x 对应位相同的符号（**check0** 对应 0，**check1** 对应 1）。如果找到，将其改写为 Y ，然后进入 **go_back** 状态准备下一轮比较；如果找到的是另一个符号或空白，则拒绝。
- **go_back**: 向左移动，目标是回到 x 部分下一个待比较的位。它会跳过 Y 和 $\#$ ，当遇到 X 时，说明回到了 x 区域的起点，此时右移一格（回到 **start** 状态）开始处理下一个 X 右边的位（即 x 的下一位）。如果意外遇到空白，说明逻辑错误，拒绝。
- **check_y_end**: 当 **start** 状态遇到 $\#$ （ x 已处理完）时进入。它向右检查，应该只能看到 Y （ y 中已匹配的位）。当遇到空白时，说明 y 也恰好处处理完，接受。如果遇到 0 或 1，说明 y 比 x 长，拒绝。

答案展示

```
input: '100#100' # 测试
blank: ' '
start state: start
```

```
table:
  start:
    0: {write: X, R: check0}
    1: {write: X, R: check1}
    '#': {R: check_y_end}
    ' ': {R: accept} # 都空则接受
  check0:
    0: R
    1: R
    Y: R # 已标记处理过则可跳过
    '#': {R: verify0}
  check1:
    0: R
    1: R
    Y: R # 已标记处理过则可跳过
    '#': {R: verify1}
  verify0:
    0: {write: Y, L: go_back}
    1: {R: reject}
    ' ': {R: reject}
    Y: R # 继续找0
  verify1:
    1: {write: Y, L: go_back}
    0: {R: reject}
    ' ': {R: reject}
    Y: R # 继续找1
  go_back:
    Y: L
    '#': L
    X: {R: start}
    0: L
    1: L
    ' ': {R: reject}
  check_y_end:
    Y: R # 继续检查Y
    ' ': {R: accept} # Y结束, 匹配
    0: {R: reject} # 超出, 拒绝
    1: {R: reject} #
  accept:
  reject:
```

题目 2

题目

设语言 $L = \{0^n : n \text{ 是斐波那契数}\}$ 。设计一个判定 L 的图灵机。

讲解

斐波那契数列定义为: $F_0 = 0, F_1 = 1, F_2 = 1, F_3 = 2, F_4 = 3, F_5 = 5, \dots$, 即 $F_n = F_{n-1} + F_{n-2}$ ($n \geq 2$)。L 中包含 $0^{F_2}(0), 0^{F_3}(00), 0^{F_4}(000), 0^{F_5}(00000), \dots$ 等字符串。我们需要判断输入的 0 串长度是否在这个数列中。

核心思路 (模拟斐波那契数列生成与比较): 我们不能直接计算 n 是否为斐波那契数, 因为图灵机无法进行复杂的算术运算。但我们可以用动态标记的方法来模拟:

1. 初始化两个“指针”: 在纸带上用两个特殊标记 (比如 **x**) 来代表当前考虑的两个连续的斐波那契数 F_k 和 F_{k+1} 在输入串中的“结束位置”。初始时, 第一个 **x** 标记第一个 0, 第二个 **x** 标记第二个 0。注意, 这里我们把输入串的起始位置与斐波那契数的计数对应起来。
2. 迭代检查与移动:
 - 比较当前输入串的长度是否等于第二个 **x** 的位置 (即当前考虑的 F_{k+1})。如果输入恰好结束在第二个 **x** 处, 当然就要接受。
 - 如果输入串的长度超过了第二个 **x**, 说明 $n > F_{k+1}$, 那么我们需要计算下一个斐波那契数 $F_{k+2} = F_k + F_{k+1}$, 并更新两个“指针”。
 - 如何计算下一个位置: 将第一个 **x** 向右移动到第二个 **x** 的当前位置, 这代表了旧的 F_{k+1} 变成了新的 F_k 。(同时要设法删掉旧的第一个 **x**) 然后, 从新的第一个 **x** (即旧的第二个 **x**) 位置出发, 向右“数”出与旧的 F_k (即旧的第一个 **x** 到纸带起点的距离) 相等的 0 的个数, 并在那里放置新的第二个 **x**, 这代表了新的 F_{k+1} 。这个“数”的过程通过状态转移和来回移动实现。重复此过程。
3. 边界与拒绝:
 - 如果输入串长度在两个连续的斐波那契数位置之间 (即超过第一个 **x** 但未达到第二个 **x** 就遇到空白), 则拒绝。
 - 如果输入是空串 ($n = 0$), 根据定义 0 不是斐波那契数, 不过以题目的具体要求为准即可。

状态设计与转移逻辑概要:

- **start**: 初始化。标记前两个 0 为 **x**, 建立初始的“指针”。
- **check_match**: 位于第二个 **x** (F_{k+1} 的边界)。检查前方是否是空白。若是, 说明输入长度正好是斐波那契数, 接受。若不是, 则标记下一个 0 为 **x** (这并不是说下一个位置也是斐波那契数, 而是为了做标记, 以协助继续计算与查找), 并进入状态 **find_the_first_x**, 开始计算下一个斐波那契数的位置。
- **check_excess**: 位于非斐波那契的边界。检查前方是否是空白。若是, 说明输入长度不是斐波那契数, 拒绝。若不是, 同样标记并进入计算流程。
- **find_the_first_x, find_the_second_x, judge, suspect1, suspect2, trust1, trust2**: 这些状态共同协作, 完成复杂的“移动第一个 **x**”和“测量距离以放置新的第二个 **x**”的任务。它们通过左右来回移动、临时改变标记、判断是否到达正确位置等操作, 模拟斐波那契数列的递推过程。其中, 代码中的 **trust** 和 **suspect** 路径分别对应了输入长度正好等于和大于当前斐波那契数边界两种情况的不同处理逻辑。具体理解可见代码注释。

答案展示

```
input: '0000000000000' # 以13为例测试
blank: ' '
```

```

start state: start
table:
  start:
    ' ': {R: accept} # 空输入, n=0, 接受
    0: {write: X, R: init_second} # 标记第一个0, 初始化第二个斐波那契数
  init_second:
    ' ': {R: accept} # 只有一个0, n=1, 接受
    0: {write: X, R: check_match} # 标记第二个0, 进入检查匹配状态 (因为2本身是斐波那契数, 应当进入match而非excess)

# 当前是一个斐波那契的位置
  check_match:
    ' ': {R: accept} # 无剩余0, 匹配斐波那契数, 接受
    0: {write: X, L: find_the_first_x} # 说明数更长, 还要继续往右读 (这里写X并不是因为是斐波那契数, 而是为了标记现在)
# 当前是一个非斐波那契的位置
  check_excess:
    ' ': {R: reject} # 超出输入长度, 拒绝
    0: {write: X, L: find_the_first_x} # 说明数更长, 还要继续往右读

# 第一个x表示当前存着的最大的斐波那契数
  find_the_first_x:
    0: L # 向左移动跳过0
    X: {L: find_the_second_x} # 找第二个x
# 第二个x表示当前最左x标识符, 实际代表右边是否达到斐波那契数
  find_the_second_x:
    0: L # 继续左移
    X: {write: 0, L: judge} # 将X恢复为0, 准备右移

# 关键区分
# trust向右行进目的是进行判断是否accept
# suspect向右行进目的是进行判断是否reject
  judge:
    ' ': {R: trust1} # 到达最左的同时, 说明右边也已经右移到了斐波那契的位置
    0: {write: X, R: suspect1} # 还没有到达最左, 说明右边还没达到斐波那契的位置
  suspect1:
    0: R # 向右移动跳过0
    X: {R: suspect2} # 遇到X, 进入右移状态2
  suspect2:
    0: R # 继续右移
    X: {write: 0, R: check_excess} # 将X恢复为0, 检查是否超出
  trust1:
    0: R # 向右移动跳过0
    X: {R: trust2} # 遇到X, 进入右移状态2
  trust2:
    0: R # 继续右移
    X: {R: check_match} # 遇到X, 返回检查匹配状态
accept:
reject:

```

题目 3

题目

在图灵机的定义中，读写头可以向左或向右移动。允许读写头向左、向右或保持不动的图灵机称为允许静止移动的图灵机，形式化定义为： $\delta : Q \times \Gamma \rightarrow Q \times \Gamma \times \{L, R, S\}$ 。证明如果语言 L 可以被允许静止移动的图灵机判定，那么 L 也可以被标准图灵机判定。

讲解

我们需要证明，允许静止移动 (S) 这个功能并没有增强图灵机的计算能力。任何用这种增强版图灵机能判定的语言，用原来的标准图灵机（只允许 L 和 R ）也能判定。证明方法是构造性的：给定一个允许静止移动的图灵机 M_S ，我们显式地构造一个标准图灵机 M 来模拟 M_S 的行为，使得两者判定相同的语言。

证明思路：关键在于如何用标准的左右移动 (L, R) 来模拟静止移动 (S)。一个直观的想法是：当 M_S 要执行 S 动作时，让 M 先向右移动一格，然后再向左移动一格，这样读写头就回到了原位。同时， M 需要在状态中记住它正在模拟一个 S 动作，以控制这两步移动。

形式化构造：设允许静止移动的图灵机 $M_S = (Q, \Sigma, \Gamma, \delta_S, q_0, q_{accept}, q_{reject})$ ，其中 $\delta_S : Q \times \Gamma \rightarrow Q \times \Gamma \times \{L, R, S\}$ 。

我们构造标准图灵机 $M = (Q', \Sigma, \Gamma, \delta, q'_0, q_{accept}, q_{reject})$ ，其中 q_{accept} 和 q_{reject} 与 M_S 相同。

1. 状态集： $Q' = Q \cup \{q_q^{stay} \mid q \in Q\}$ ，为每个 $q \in Q$ 加一个辅助状态 q_q^{stay} （用来完成“右移再左移”）。
2. 初始状态： $q'_0 = q_0$ 。
3. 转移函数：
 - 若 $\delta_S(q, a) = (p, b, D)$ 且 $D \in \{L, R\}$ ，则 $\delta(q, a) = (p, b, D)$ （直接复制）。
 - 若 $\delta_S(q, a) = (p, b, S)$ ，用两步模拟停留：
 - 1) $\delta(q, a) = (q_q^{stay}, b, R)$;
 - 2) 对任意 $c \in \Gamma$ ， $\delta(q_q^{stay}, c) = (p, c, L)$ 。

如此，先右移一格再左移回原格，进入同样的下一状态 p ，头位置与 M_S 一致。

正确性：通过上述构造， M 严格地模拟了 M_S 的每一步计算。对于 L 和 R 移动， M 直接复制。对于 S 移动， M 通过一系列状态和移动，最终效果也是在原位置写入了相同的符号，并且读写头停留在原位置（经过校正后），然后进入相同的下一个状态。因此，对于任何输入串， M 和 M_S 的接受/拒绝行为完全一致。由于 M 是标准图灵机，这就证明了标准图灵机的计算能力至少与允许静止移动的图灵机一样强。

复习小结：第五节课重点掌握图灵机的形式化定义（七元组）和基本运行原理，理解其作为算法形式化模型的核心地位。通过作业，练习了图灵机设计的基本方法：逐位比较、利用标记模拟递推过程。同时，通过证明题理解了图灵机模型某些定义上的变种（如允许静止移动）并不改变其根本的计算能力，证明的关键在于如何用原始操作模拟新增的操作。

第六节课复习资料

一、本节课知识点讲解

6.1 识别 (Recognizing) 与判定 (Deciding) 的区别

在图灵机的范畴内，我们需要区分两个重要概念：

- 识别 (**Recognizing**): 如果存在一个图灵机 M ，使得对于任意输入 x ，当 x 属于语言 L 时 M 最终会接受 x ，而当 x 不属于 L 时 M 要么拒绝要么永远不停机（即可能进入无限循环），那么我们说 M 识别 (**recognizes**) 了语言 L ，并且称 L 是图灵可识别的 (**Turing recognizable**)。
- 判定 (**Deciding**): 如果存在一个图灵机 M ，使得对于任意输入 x ， M 总是能在有限步内停机，并且当 $x \in L$ 时接受， $x \notin L$ 时拒绝，那么我们说 M 判定 (**decides**) 了语言 L ，并且称 L 是可判定的 (**decidable**)。

显然，每个可判定的语言都是图灵可识别的，但反过来不一定成立（例如，著名的停机问题对应的语言就是图灵可识别但不可判定的）。

6.2 可计算函数 (Computable Function)

设 $f: \{0,1\}^* \rightarrow \{0,1\}^* \cup \{\perp\}$ 是一个部分函数（即某些输入上可能没有定义，用 $\perp$ 表示）。我们称一个图灵机 M 在时间 $T(n)$ 内计算 (**computes**) f ，如果对于每个输入 x （且 $f(x) \neq \perp$ ）， M 在至多 $T(|x|)$ 步内停机，并且此时纸带上除了空白符外剩下的内容正好是 $f(x)$ 。

6.3 图灵机的变体及其等价性 (重点)

为什么图灵机是一个好的计算模型定义？一个重要原因是它的稳健性 (**robustness**)：许多看起来增强或修改了图灵机定义得到的变体，在计算能力上并没有超过标准图灵机（即它们能判定的语言类是一样的），而且这些变体之间的运行时间也只差一个多项式因子。这使得图灵机模型具有很强的通用性。

变体 1: 字母表改变 结论：如果语言 L 能被一个字母表为 Γ 的图灵机在时间 $T(n)$ 内判定，那么它也能被一个字母表为 $\{0,1,\text{blank}\}$ 的图灵机在时间 $O(\log |\Gamma| \cdot T(n))$ 内判定。

证明思路：将 Γ 中的每个符号用 $k = \lceil \log |\Gamma| \rceil$ 个二进制位进行编码。模拟原图灵机的一步时，需要读取 k 位来解码原符号，然后根据转移函数确定要写入的新符号（也是 k 位），再移动读写头。因此，一步模拟需要 $O(k)$ 步，总时间就是 $O(k \cdot T(n))$ 。

变体 2: 单向图灵机 (One-way Turing Machine) 这种图灵机的纸带有一个左边界（不能向左越过），并且规定在左边界上只能向右移动。

结论：单向图灵机与标准图灵机计算能力等价，且时间开销只差常数倍。

证明思路：将标准图灵机的双向无限纸带“折叠”到单向纸带上。具体地，将标准纸带上的位置 $0, 1, 2, \dots$ 映射到单向纸带的位置 $0, 2, 4, \dots$ ，而将位置 $-1, -2, -3, \dots$ 映射到单向纸带的位置 $1, 3, 5, \dots$ 。这样，模拟一步标准移动时，单向图灵机需要移动两格（因为相邻的两个逻辑位置在物理位置上可能相隔 2）。同时，单

向图灵机需要用状态记住当前读写头是在“正半轴”还是“负半轴”，以决定移动方向。因此，一步模拟需要 $O(1)$ 步。

变体 3: 多带图灵机 (Multi-tape Turing Machine) 一个 k 带图灵机有 k 条独立的纸带，每条纸带都有自己的读写头，转移函数同时基于所有读写头下的符号，并决定每个头写入的符号和移动方向。

结论：任何 k 带图灵机在时间 $T(n)$ 内判定的语言，也能被一个标准单带图灵机在时间 $O(T(n)^2)$ 内判定。

证明思路：这是本节课的重点，也是作业题的核心思想。我们用一条纸带模拟 k 条纸带。具体方法是：

- 将 k 条纸带的内容交错存储：即单带上的位置依次存储第 1 条纸带的第 1 个符号、第 2 条纸带的第 1 个符号、……、第 k 条纸带的第 1 个符号，然后是第 1 条纸带的第 2 个符号、第 2 条纸带的第 2 个符号……以此类推。
- 为了标记每个纸带的读写头位置，我们为字母表中的每个符号 a 引入一个带“帽”的版本 $\hat{a}$ ，表示该位置对应纸带的读写头正指向这个符号。
- 模拟一步多带图灵机的操作时，标准图灵机需要扫描整个已使用的纸带区域，收集所有 k 个读写头下的符号（通过寻找带帽的符号），然后根据原转移函数计算出新的状态、要写入的符号以及每个头的移动方向，然后它就可以再次扫描纸带来更新这些带帽符号（移动“帽”的位置）了。由于 k 是常数，一次扫描需要 $O(T(n))$ 时间。因此，模拟一步需要 $O(T(n))$ 时间，总共 $T(n)$ 步，总时间就是 $O(T(n)^2)$ 。

注意：平方时间是较松的界限，实际可优化到 $O(T(n) \log T(n))$ ，但课程中通常只要求掌握平方模拟。

变体 4: RAM 图灵机 (Random Access Memory Turing Machine) 这种图灵机除了常规纸带外，还有一个通过地址随机访问的“内存”（一个无限数组）。它有一条特殊的地址带，用于指定要读写内存的哪个位置。

结论：如果语言 L 能被一个 RAM 图灵机在时间 $T(n)$ 内判定，那么它也能被一个多带图灵机在时间 $O(T(n)^3)$ 内判定（如果地址长度有界，则只需 $O(T(n)^2)$ 时间）。

证明思路：用一条额外的纸带来模拟内存数组，存储 (地址, 值) 对。当需要访问某个地址 i 时，模拟器扫描这条纸带寻找地址为 i 的项。每次 RAM 要读写某个地址时，图灵机的读写头必须在带子上线性扫描来寻找这个地址。由于带子上存储的记录总数最多可达 $T(n)$ ，每次寻道在最坏情况下需要移动 $O(T(n))$ 步。此外，在单位代价模型下，RAM 的一个单元可以存放非常大的数，且对其进行算术运算只需 1 步。但在 $T(n)$ 步中，通过不断的加法或位移，RAM 寄存器中的数值长度（位数）最高可以增长到 $O(T(n))$ 位。因此标准图灵机模拟时又增加了一重 $O(T(n))$ 。总共 $(T(n))$ 次访问，总时间就是 $O(T(n)^3)$ 。

6.4 复杂性类 P 的定义

基于多带图灵机，我们可以定义时间复杂度类：

设 $T: \mathbb{N} \rightarrow \mathbb{N}$ 是一个函数。语言 L 属于 $\mathbf{DTIME}(T(n))$ 当且仅当存在一个常数 c 和一个多带图灵机 M ，使得 M 在时间 $c \cdot T(n)$ 内判定 L 。

多项式时间类 P 定义为所有多项式时间可判定的语言的集合：

$$P = \bigcup_{c \geq 1} \mathbf{DTIME}(n^c)$$

也就是说，一个语言 L 属于 P ，当且仅当存在一个多带图灵机和一个常数 k ，使得该图灵机能在 $O(n^k)$ 时间内判定 L 。

重要备注：由于上述各种图灵机变体之间模拟只产生多项式时间开销，所以 P 的定义不依赖于具体使用哪种图灵机模型（标准单带、多带、RAM 等）。这是 P 类作为“易解问题”类的一个稳健性保证。

6.5 图灵机的可数性

集合的可数（**countable**）是指其元素可以与自然数一一对应（即存在一个从自然数到该集合的单射）。有限集和可数无限集都是可数的。

结论：所有图灵机（包括各种变体）的集合是可数的。

证明思路：每个图灵机都可以用一个有限的字符串（例如它的形式化描述）来编码。因为有限字母表上的所有有限字符串的集合是可数的（可以按长度和字典序枚举），所以图灵机的集合作为其子集也是可数的。这个结论非常重要，它意味着存在“不可计算”的函数（因为所有函数的集合是不可数的），从而引出计算理论中的许多深刻结果。

二、作业题目详细讲解

题目 1

题目

定义一个二维图灵机为每个磁带都是无限网格的图灵机（机器不仅可以左右移动，还可以上下移动）。证明如果语言 L 可以被二维图灵机在时间 $T(n)$ 内判定，那么 $L \in \text{DTIME}(T(n)^2)$ 。

讲解

我们只需给出直接的模拟方案：在 $T(n)$ 步内，二维头最多触及 $O(T(n))$ 行和 $O(T(n))$ 列，共 $O(T(n)^2)$ 个格子；单带模拟每步扫描这一活动区即可，单步成本 $O(T(n)^2)$ ：用分块/行分隔让一次扫描成本 $O(T(n))$ ，于是总时间 $O(T(n)^2)$ 。下面给出简明构造。

构造：

1. 布局：在一维纸带上按行序存储有限的“活动矩形”：先存第 1 行的已访问列，紧跟一个行分隔符，再存第 2 行，依此类推；每个单元记录其符号，并用“帽符号”标记当前二维头所在单元。
2. 头位置记录：帽符号唯一地出现在某个单元上，表示当前二维坐标；无需显式存 (x, y) ，只靠帽子定位。
3. 模拟一步：
 - 从纸带起点扫一遍，找到帽符号，得到当前符号。
 - 依据二维机的转移，写新符号、决定方向；在扫的过程中把帽子移到目标单元（如果目标在同一行则在当前行中左/右移，否则越过分隔符到上一行或下一行；理论上此时必能到达目标的行/列，否则说明此位置尚未被构造，则在相应位置插入空白单元并放置帽子）。
 - 该扫描长度与活动矩形的周长线性相关，为 $O(T(n))$ 。
4. 时间界：总步数 $T(n)$ ，每步 $O(T(n))$ 扫描/更新，故总 $O(T(n)^2)$ 。

结论：因此，任何被二维图灵机在 $T(n)$ 时间内判定的语言 L ，都存在一个标准图灵机在 $O(T(n)^2)$ 时间内判定它，即 $L \in \text{DTIME}(T(n)^2)$ 。

题目 2

题目

定义一个单带双头图灵机为只有一个磁带但有两个独立读写头的图灵机。每个头可以同时独立地读、写和移动。两个头可以同时读取同一个磁带单元。冲突解决如下：

- 如果两个头试图同时写入同一个单元，第一个头先写，第二个头覆盖之前的内容。
- 如果第一个头试图写而第二个头试图读同一单元，第一个头先写，第二个头读取更新后的内容。
- 如果第一个头试图读而第二个头试图写同一单元，第一个头先读，读取旧内容，然后第二个头更新内容。

证明如果语言 L 可以被单带双头图灵机在时间 $T(n)$ 内判定，那么 $L \in \text{DTIME}(T(n)^2)$ 。

讲解

核心：用单头 TM 的三轨纸带模拟双头单带机。（三轨和标准是等价的）

构造要点：

1. 三轨布局：轨道 1 存内容；轨道 2 用 H_1 标记头 1 位置；轨道 3 用 H_2 标记头 2 位置。帽子唯一。
2. 模拟一步：
 - 先左到右扫描一次，找到 H_1, H_2 ，读出对应符号 a_1, a_2 ，在状态中记下，按双头机转移算出 (q', b_1, b_2, D_1, D_2) 。
 - 再扫一遍按规则写与移：先执行头 1 的写/移，再执行头 2 的读/写/移，若同格写则头 2 覆盖（这样就可以自然地实现冲突规则）。移动帽子到新格；若需扩展纸带则插空白。
3. 时间：已用长度 $O(T(n))$ ，每步两次扫描 $O(T(n))$ ，共 $T(n)$ 步，时间 $O(T(n)^2)$ 。

结论：因此， L 可以被标准图灵机在 $O(T(n)^2)$ 时间内判定，即 $L \in \text{DTIME}(T(n)^2)$ 。

题目 3

题目

证明多带图灵机的数量是可数的。

讲解

我们需要证明所有多带图灵机（包括不同带数、不同状态数、不同转移函数等等）构成的集合是一个可数集。可数集是指其元素可以与自然数建立一一对应（或者至少存在一个从自然数到该集合的满射，即可以列出来）。

证明思路：

每个图灵机都可以用一个有限的字符串来完全描述（因为它的组件：状态集、字母表、转移函数等都是有限

的)。而所有有限字符串的集合是可数的。因此，图灵机的集合是可数集的一个子集，从而也是可数的。

具体证明：

1. 图灵机的有限描述：

一个 k 带图灵机 (k 是正整数) 由七部分组成 (上述已述)。

所有这些组件都是有限的 (除了 k 本身, 但 k 是正整数, 也是可数的)。因此, 我们可以将整个图灵机编码成一个字符串。

2. 编码为字符串：

我们可以设计一种编码方案, 将上述组件表示为一个二进制字符串。例如：

- 用二进制数字表示 k 。
- 将状态和符号用二进制串编号。
- 转移函数可以用一个列表表示, 列表中的每一项是 $(q, a_1, \dots, a_k, q', b_1, \dots, b_k, D_1, \dots, D_k)$ 的形式。
- 整个描述可以拼接起来, 并用特殊分隔符隔开。

重要的是, 每个图灵机都有多种编码方式, 但至少存在一种编码。我们固定一种编码方案。

3. 可数性论证：

- 所有有限二进制字符串的集合 $\{0, 1\}^*$ 是可数的。这是因为我们可以按长度和字典序枚举它们：先枚举长度为 0 的字符串 (空串), 然后长度为 1 的字符串 (0,1), 长度为 2 的字符串 (00,01,10,11), 等等。
- 每个多带图灵机都对应至少一个二进制字符串 (它的编码), 而不同的图灵机一定有不同编码。因此, 所有多带图灵机的集合与一个二进制字符串集合的子集一一对应。由于可数集的子集也是可数的, 所以多带图灵机的集合是可数的。

4. 处理不同的带数：

注意, 多带图灵机包括 $k = 1, 2, 3, \dots$ 所有可能。每个固定的 k , k 带图灵机的集合是可数的 (如上所述)。可数多个可数集的并集仍然是可数的 (因为我们可以用“对角线法”枚举所有并集中的元素)。因此, 所有多带图灵机的集合是可数的。

结论：多带图灵机的数量是可数的。

复习小结：第六节课重点掌握图灵机各种变体 (二维、双头、多带、RAM 等) 与标准图灵机的等价性, 特别是如何用标准图灵机模拟这些变体并分析时间开销 (通常是平方或多项式时间)。同时, 理解图灵机的可数性证明, 这为后续讨论不可计算性问题 (如停机问题) 奠定了基础。作业题目要求我们具体构造模拟并分析时间, 需要熟练掌握编码和模拟的基本技巧。

第七节课复习资料

一、本节课知识点讲解

7.1 Church-Turing 论题与强 Church-Turing 论题

- **Church-Turing 论题**：该论题认为，任何在物理世界中可计算的函数，都可以被一个图灵机计算。这是一个哲学命题，无法被严格证明，但被计算机科学界广泛接受。它为“什么是可计算的”提供了标准答案。
- **强 Church-Turing 论题**：该论题是更强的版本，它认为任何在物理世界中可计算的函数，都可以被一个图灵机在多项式时间内计算。这一论题争议很大，很多人不相信，因为现实中可能存在物理过程（如量子计算）能在多项式时间内解决图灵机需要指数时间的问题，因此图灵机不是“天花板”。

重点：这两个论题的核心是理解图灵机作为计算能力上限（Church-Turing 论题）和时间效率上限（强 Church-Turing 论题）的参考模型。题目较少涉及，但理解其是理解“可计算性”和“复杂性类 P”的基础。

7.2 通用图灵机 (Universal Turing Machine, UTM)

通用图灵机是一个特殊的图灵机 U ，它的输入是一个对 (α, x) ，其中 α 是另一个图灵机 M_α 的描述（编码）， x 是输入字符串。 U 的行为是模拟 M_α 在输入 x 上的运行。

- 为什么需要 **UTM**？：这实现了冯·诺依曼架构的“存储程序”思想——硬件（UTM）是固定的，通过改变软件（编码 α ）来完成不同的计算任务。
- 编码：每个图灵机 M 都可以被编码为一个二进制串 $\alpha \in \{0, 1\}^*$ ，记作 M_α 。每个图灵机有无穷多种编码。
- 时间复杂度：如果被模拟的图灵机 M_α 在 T 步内停机，那么一个精心构造的 UTM 可以在 $O(T \log T)$ 时间内完成模拟。如果采用更简单的模拟方法，时间复杂度可能是 $O(T^2)$ 。

7.3 复杂性类 P

这是本节课和作业的绝对核心。

- 定义：语言 L 属于 **P**，当且仅当存在一个（确定型）图灵机 M 和一个常数 k ，使得 M 能在 $O(n^k)$ 时间内判定 L 。
 - “判定”意味着对任何输入， M 总是能在多项式步数内停机，并给出接受或拒绝的正确回答。
- 关键性质（稳健性）：**P** 的定义不依赖于图灵机的具体变体（单带、多带、RAM 等）。因为不同变体之间的模拟只会带来多项式时间的开销。这使得 **P** 作为一个“易处理问题”的类是非常稳健的。
- 封闭性：**P** 类语言在某些运算下是封闭的，即对 **P** 中的语言进行这些运算后，得到的新语言仍然在 **P** 中。

学习目标：必须熟练掌握 **P** 的定义，并能够运用定义，通过构造新的多项式时间图灵机来证明其封闭性。

7.4 利用空间限制判定行为

当一台图灵机的运行空间被限制为一个已知函数 $f(n)$ （如 n^2 ）时，即使它的运行时间可能无限长，其可能出现的全局状态（configuration）总数也是有限的。

一个全局状态包括：1. 当前控制状态。2. 读写头在有限带区上的位置。3. 有限带区上每个格子的内容。

如果图灵机在有限空间内运行且永远不停机，根据鸽巢原理，它必然在有限步内重复某个全局状态，从而进入一个无限循环。

判定算法：因此，要判定一台空间受限的图灵机是否“永远循环”，我们可以模拟它运行有限步（步数上限 $>$ 可能的全局状态总数）。如果在限制步数内它停机了，那它就不是永远循环；如果超过了限制步数还没停机，那它一定已经进入了循环（因为状态已经重复）。

二、作业题目详细讲解

题目 1

题目

证明 $\mathbf{P}$ 类语言在下列操作下是封闭的：(a) 并集 (b) 交集 (c) 连接。

讲解

我们需要证明：如果语言 L_1 和 L_2 都属于 $\mathbf{P}$ ，那么 $L_1 \cup L_2, L_1 \cap L_2, L_1 \circ L_2$ （连接）也属于 $\mathbf{P}$ 。

核心思路：根据 $\mathbf{P}$ 的定义，存在多项式时间图灵机 M_1 判定 L_1 ， M_2 判定 L_2 。我们的任务是针对每种运算，构造一个新的图灵机 M ，它能利用 M_1 和 M_2 作为子程序，并在多项式时间内完成判定。

(a) 并集封闭性

- 目标：判定输入 x 是否属于 $L_1 \cup L_2$ 。即， $x \in L_1$ 或 $x \in L_2$ 。
- 构造判定机 M ：
 1. 在输入 x 上，先运行判定 L_1 的图灵机 $M_1(x)$ 。
 2. 如果 $M_1(x)$ 接受，则 M 立刻接受（因为 x 已经属于 L_1 ，自然属于并集）。
 3. 如果 $M_1(x)$ 拒绝，则接着运行判定 L_2 的图灵机 $M_2(x)$ 。
 4. M 的输出与 $M_2(x)$ 相同： M_2 接受则 M 接受， M_2 拒绝则 M 拒绝。
- 时间复杂度分析：
 - 设 M_1 的运行时间为 $p_1(|x|)$ ， M_2 的运行时间为 $p_2(|x|)$ ，其中 p_1 和 p_2 是多项式。
 - M 在最坏情况下（ M_1 拒绝）需要先运行 M_1 ，再运行 M_2 ，总时间为 $p_1(|x|) + p_2(|x|)$ 。两个多项式之和仍是多项式。因此 M 在多项式时间内运行。
- 结论： $L_1 \cup L_2 \in \mathbf{P}$ 。

(b) 交集封闭性

- 目标：判定输入 x 是否属于 $L_1 \cap L_2$ 。即， $x \in L_1$ 且 $x \in L_2$ 。
- 构造判定机 M ：
 1. 在输入 x 上，先运行 $M_1(x)$ 。
 2. 如果 $M_1(x)$ 拒绝，则 M 立刻拒绝（因为 x 连 L_1 都不属于，肯定不属于交集）。

3. 如果 $M_1(x)$ 接受, 则接着运行 $M_2(x)$ 。

4. M 的输出与 $M_2(x)$ 相同。

• 时间复杂度分析: 与并集情况完全一样, 总时间为 $p_1(|x|) + p_2(|x|)$, 是多项式。

• 结论: $L_1 \cap L_2 \in P$ 。

(c) 连接封闭性

• 目标: 判定输入字符串 x 是否属于 $L_1 \circ L_2 = \{yz \mid y \in L_1, z \in L_2\}$ 。即, 是否存在一种方式把 x 切成两段 y 和 z , 使得 $y \in L_1$ 且 $z \in L_2$ 。

• 构造判定机 M :

设输入 x 的长度为 n 。

1. M 会尝试所有可能的切分方式。对于每一个 i 从 0 到 n :

– 令 y 为 x 的前 i 个字符, z 为剩下的 $n - i$ 个字符。(注意: $i = 0$ 时 y 是空串, $i = n$ 时 z 是空串)

– 模拟运行 $M_1(y)$ 和 $M_2(z)$ 。

– 如果 $M_1(y)$ 和 $M_2(z)$ 都接受, 那么 M 立即接受 x 。

2. 如果尝试了所有 $n + 1$ 种切分后, 没有一种能使 M_1 和 M_2 同时接受, 则 M 拒绝 x 。

• 时间复杂度分析:

– 共有 $n + 1$ 种切分。对于每种切分, 需要运行 M_1 和 M_2 各一次。一次模拟 $M_1(y)$ 的时间不超过 $p_1(n)$ (因为 $|y| \leq n$), 一次模拟 $M_2(z)$ 的时间不超过 $p_2(n)$ 。

– 因此, 处理一种切分的总时间不超过 $p_1(n) + p_2(n)$ 。

– M 的总运行时间不超过 $(n + 1) \times (p_1(n) + p_2(n))$ 。因为 n 是多项式, $(p_1(n) + p_2(n))$ 是多项式, 它们的乘积仍是多项式。

• 结论: $L_1 \circ L_2 \in P$ 。

总结: 证明 P 类封闭性的通用模式是: 假设存在多项式时间判定机, 然后构造一个新的判定机去调用它们。关键在于论证新判定机的运行时间仍然是输入长度 n 的多项式函数。

题目 2

题目

设 $L = \{(\alpha, x) : \text{在输入 } x \text{ 上, 图灵机 } M_\alpha \text{ 最多使用 } n^2 \text{ 空间, 并且永远循环}\}$ 。证明 L 是可判定的。

讲解

我们需要证明存在一个图灵机 D , 对于任意输入 (α, x) , D 总能停机, 并正确判断 (α, x) 是否属于 L 。

理解题意:

• M_α : 由字符串 α 编码的图灵机。

- “最多使用 n^2 空间”： $n = |x|$ 是输入长度。 M_α 在输入 x 上运行时，其读写头访问过的纸带格子数不超过 n^2 。
- “永远循环”： M_α 在输入 x 上永不停机（既不到达接受状态，也不到达拒绝状态）。

证明核心思想：由于 M_α 被限制在 n^2 个格子内运行，即使它可能运行无限步，其可能出现的不同全局状态 (Configuration) 总数是有限的。一个全局状态由三部分决定：

1. 当前状态： M_α 的有限控制状态，设状态数为 Q （由 α 决定）。
2. 读写头位置：在 n^2 个格子中，头的位置有 n^2 种可能。
3. 纸带内容：每个格子可以写 Γ 个字母表符号中的一个（ Γ 的大小由 α 决定）。 n^2 个格子的内容组合最多有 $|\Gamma|^{n^2}$ 种。

因此，所有可能的全局状态总数 $C \leq Q \times n^2 \times |\Gamma|^{n^2}$ 。这是一个有限的数（虽然很大）。因此，超过 C 步未停机必然重复状态，从而进入循环。模拟超过 C 步还未停机即可判定“永远循环”。

构造判定机 D （输入 (α, x) ，令 $n = |x|$ ）：

1. 解析 α 得到状态数 Q 、字母表大小 $|\Gamma|$ ，计算 $C = Q \times n^2 \times |\Gamma|^{n^2}$ 。
2. 模拟 M_α ：
 - 监视空间：若头超出距离起点 n^2 的范围，立即拒绝。
 - 监视步数：若步数达到 $C + 1$ 仍未停机，立即接受（已进入循环）。
 - 停机检查：若 M_α 在界内停机（接受或拒绝），立即拒绝。

由于对于所有输入， D 都按照上述有穷步骤执行并给出回答，因此 D 是一个判定 L 的图灵机。所以 L 是可判定的。

总结：利用“有限空间，有限状态，可探测循环”，把“永远循环”判定化为有限步模拟。

第八节课复习资料

一、本节课知识点讲解

8.1 不可判定性的证明技术：对角化法

- 对角化法：一种证明不可判定性的经典方法。其核心思想是：假设所有图灵机（即可计算函数）可以列成一个表，然后通过构造一个“对角线”上的语言，使得它与表中每一行（即每个图灵机识别的语言）都不同，从而证明存在无法被任何图灵机判定的语言。
- 典型应用：证明 $L_{flip} = \{\alpha \mid M_\alpha \text{ 不接受 } \alpha\}$ 是不可判定的。证明思路是：假设存在判定机 M_β ，则考虑 β 自身是否属于 L_{flip} ，会导出矛盾。

8.2 通用图灵机 (UTM) 的时间复杂度 (回顾)

- 定理 4.1：存在一个（三带）通用图灵机 U ，使得对于任意编码 α 和输入 x ， $U(\alpha, x)$ 模拟 $M_\alpha(x)$ ，且如果 M_α 在 T 步内停机，则 U 在 $O_\alpha(T \log T)$ 步内停机。
- 证明思路：

1. 用单带模拟多带图灵机，通过引入缓存区（大小为 2^{i+1} 的块 L_i 和 R_i ）来管理纸带内容的移动。
 2. 维护不变量：每个缓存区要么满、要么半满、要么空，且 L_i 和 R_i 的已用空间之和为 2^{i+1} 。
 3. 每次读写头移动时，通过调整这些缓存区的内容来实现，摊还代价为 $O(\log T)$ 。
- 重要性：这是理解为什么“多项式时间模拟”可行的基础之一。

8.3 停机问题及其归约

- 停机问题： $L_{halt} = \{(\alpha, x) \mid M_\alpha \text{ 在输入 } x \text{ 上停机}\}$ 。
- 不可判定性证明：通过归约到 L_{flip} 来证明。假设 L_{halt} 可判定，则可以构造一个判定 L_{flip} 的图灵机，矛盾。
- 归约（**Reduction**）：证明问题 A 不可判定的常用方法。假设 A 可判定，利用 A 的判定机作为子程序去解决一个已知不可判定的问题（如停机问题），从而导出矛盾。

8.4 其他不可判定问题

- 接受问题： $L_{accept} = \{(\alpha, x) \mid M_\alpha \text{ 接受 } x\}$ 。
- 语言等价问题： $L_{equal} = \{(\alpha, \beta) \mid L(M_\alpha) = L(M_\beta)\}$ 。
- 正则性问题： $L_{regular} = \{\alpha \mid L(M_\alpha) \text{ 是正则语言}\}$ 。
- 证明方法：通常都是从停机问题（或接受问题）归约。构造一个特殊的图灵机，其语言的性质（如是否正则、是否等于某个固定语言）取决于另一个图灵机在某个输入上是否停机（或接受）。

8.5 Rice 定理

- **Rice 定理**：任何关于图灵机语言的非平凡性质都是不可判定的。
 - “性质”指一个语言集合 $P \subseteq \{0, 1\}^*$ ，且满足：如果 $L(M_\alpha) = L(M_\beta)$ ，则 $\alpha \in P \iff \beta \in P$ 。即性质只依赖于图灵机识别的语言，而不是具体的机器。
 - “非平凡”指 P 既不是空集，也不是所有图灵机编码的集合。
- 应用：直接证明许多问题（如语言是否为空、是否为有限集、是否包含某个特定串、是否为正则语言等）都是不可判定的，无需单独归约。
- 注意：作业中出现的要求不使用 Rice 定理的题目，目的是练习归约构造。

8.6 可判定与不可判定的对比

- 可判定问题举例：有限步停机问题（ $L = \{(\alpha, x, k) \mid M_\alpha \text{ 在输入 } x \text{ 上在 } k \text{ 步内停机}\}$ ）。因为 k 是有限的，可以直接模拟 k 步。
- 关键区别：问题是否涉及对无限行为的判定（如“是否永远循环”、“语言是否无穷”），通常需要探索无限的可能性，往往不可判定；而如果限制在有限资源（如时间步数、空间大小）内，则通常可判定。

二、作业题目详细讲解

题目 1

题目

设 $L = \{(\alpha, \beta) : \text{图灵机 } M_\alpha, M_\beta \text{ 满足 } L(M_\alpha)^* = L(M_\beta)\}$ 。证明 L 是不可判定的。

讲解

目标：归约停机问题，构造 M_α 和 M_β ，使 $(\alpha, \beta) \in L$ 当且仅当 $M(w)$ 停机。

构造方法：

定义 M_α ：

- 输入 x 。
- 模拟 M 在 w 上运行。
- 如果 M 在 w 上不停机，则 M_α 永远循环，不接受任何 x ，故 $L(M_\alpha) = \emptyset$ 。
- 如果 M 在 w 上停机，则 M_α 接受所有 x （即 $L(M_\alpha) = \Sigma^*$ ）。

定义 M_β 使得 $L(M_\beta) = \Sigma^*$ 。注意 Σ^* 的克林闭包还是 Σ^* （因为 Σ^* 已经包含所有串，且对连接封闭）。

现在分析：

- 如果 M 在 w 上停机，则 $L(M_\alpha) = \Sigma^*$ ，那么 $L(M_\alpha)^* = (\Sigma^*)^* = \Sigma^* = L(M_\beta)$ 。
- 如果 M 在 w 上不停机，则 $L(M_\alpha) = \emptyset$ ，那么 $L(M_\alpha)^* = \emptyset^* = \{\varepsilon\}$ ，而 $L(M_\beta) = \Sigma^*$ ，显然 $\{\varepsilon\} \neq \Sigma^*$ 。

因此， $(\alpha, \beta) \in L$ 当且仅当 M 在 w 上停机。

结论：如果我们能判定 L ，就能判定停机问题。但停机问题不可判定，所以 L 不可判定。

题目 2

题目

设 $L = \{\alpha : L(M_\alpha) \text{ 包含至少两个素数}\}$ ，其中机器 M_α 的每个输入 $x \in \{0, 1\}^*$ 被解释为一个二进制数（最高位在前）。证明 L 是不可判定的。（不要使用 Rice 定理）

讲解

我们需要证明 L 是不可判定的。这里要求不使用 Rice 定理，所以必须构造一个从某个不可判定问题（如停机问题）到 L 的归约。

理解题意：

- M_α 是一个图灵机，其输入 x 是一个二进制字符串，被解释为一个二进制数（例如字符串 "101" 解释为数字 5）。
- $L(M_\alpha)$ 是 M_α 接受的所有二进制字符串的集合，也就是它接受的所有二进制数的集合。
- L 是所有这样的编码 α 的集合： M_α 接受的语言（这里的语言实际就是二进制数）中包含至少两个素数。

证明思路：

假设 L 是可判定的，即存在一个图灵机 D 能够判定 L 。我们将利用 D 来构造一个判定停机问题的图灵机，从而产生矛盾。

具体构造 M_α ：

M_α 的行为如下：

- 输入：一个二进制字符串 x （解释为一个整数 n ）。
- 步骤：
 1. M_α 首先模拟 M 在 w 上的运行。
 2. 如果 M 在 w 上不停机，那么 M_α 将永远模拟下去，不会进行后续步骤。因此， M_α 不会接受任何输入 x ，即 $L(M_\alpha) = \emptyset$ 。
 3. 如果 M 在 w 上停机，那么 M_α 完成模拟后，继续执行：检查 x 对应的整数 n 是否是素数（这可以在有限时间内完成）。如果 n 是素数，则 M_α 接受 x ；否则拒绝。

分析 $L(M_\alpha)$ ：

- 情况 1： M 在 w 上不停机。则 M_α 永远循环在模拟步骤中，不会接受任何 x 。所以 $L(M_\alpha) = \emptyset$ ，显然不包含任何素数。因此 $\alpha \notin L$ 。
- 情况 2： M 在 w 上停机。则 M_α 会检查每个输入 x 是否是素数，并接受所有素数。由于素数有无穷多个， $L(M_\alpha)$ 包含所有素数（的二进制表示），当然包含至少两个素数。因此 $\alpha \in L$ 。

结论： $\alpha \in L$ 当且仅当 M 在 w 上停机。

因此，如果我们有判定 L 的图灵机 D ，那么对于任意 (M, w) ，我们可以构造出对应的 α ，然后运行 $D(\alpha)$ 。如果 D 接受 α ，则 M 在 w 上停机；如果 D 拒绝 α ，则 M 在 w 上不停机。这就意味着我们解决了停机问题。但停机问题是不可判定的，矛盾。所以假设错误， L 不可判定。

关键点：

- M_α 的构造依赖于 (M, w) ，但 M_α 的输入 x 是独立的。
- 素数判定是可行的（尽管可能需要多项式时间，但在这里我们只关心可判定性，不关心效率）。
- 因为素数有无穷多个，所以当 M 停机时， $L(M_\alpha)$ 包含无穷多个素数，自然满足“至少两个”。

题目 3

题目

设 $L = \{(\alpha, x, k) : M_\alpha \text{ 在输入 } x \text{ 上在 } k \text{ 步内停机}\}$ ，其中 k 以二进制数给出（最高位在前）。证明 L 是可判定的。

讲解

我们需要证明 L 是可判定的。也就是说，存在一个图灵机 D ，对于任意输入 (α, x, k) ， D 总能停机，并正确判断 M_α 是否在输入 x 上于 k 步内停机。

证明思路：

由于 k 是一个给定的二进制数，它表示一个具体的正整数。我们可以在有限时间内模拟 M_α 在输入 x 上运

行最多 k 步。如果在 k 步内 M_α 停机了，就接受；如果运行了 k 步后还没有停机，就拒绝。

构造判定机 D ：

D 的输入为 (α, x, k) ，其中 k 是二进制字符串。

1. D 首先将二进制字符串 k 转换成对应的正整数。
2. D 模拟图灵机 M_α 在输入 x 上的运行，但最多只模拟 k 步。在模拟过程中：
 - 如果 M_α 在某个步数 ($\leq k$) 内进入了接受状态或拒绝状态（即停机），则 D 立即停止模拟，并接受 (α, x, k) 。
 - 如果模拟了 k 步后， M_α 仍未停机，则 D 停止模拟，并拒绝 (α, x, k) 。

正确性：

- 如果 $(\alpha, x, k) \in L$ ，即 M_α 在输入 x 上确实在 k 步内停机，那么 D 在模拟过程中会观察到停机，并接受。
- 如果 $(\alpha, x, k) \notin L$ ，即 M_α 在 k 步内没有停机，那么 D 模拟 k 步后未发现停机，会拒绝。

因此， D 正确地判定了 L ，所以 L 是可判定的。

复习小结：第八节课重点掌握不可判定性的证明方法，特别是通过归约从停机问题出发。要理解如何针对一个给定的性质构造图灵机，使得该性质是否成立等价于另一个图灵机在某个输入上是否停机。同时，要区分哪些问题是可判定的（通常涉及有限步模拟），哪些是不可判定的（涉及无限行为或非平凡语言性质）。作业题目练习了归约构造和可判定性证明，需要仔细体会其中的构造技巧。

第九节课复习资料

一、本节课知识点讲解

9.1 邮政对应问题 (Post Correspondence Problem, PCP)

PCP 定义： 给定一个有限的“骨牌”集合 $P = \{[t_1, b_1], [t_2, b_2], \dots, [t_k, b_k]\}$ ，其中每个 t_i 和 b_i 都是字母表 Σ 上的非空字符串。**PCP 问题是：** 是否存在一个非空的骨牌下标序列 $s_1, s_2, \dots, s_m$ ($m \geq 1$ ，且允许重复选择同一骨牌)，使得将这些骨牌顶部的字符串 $t_{s_1}, t_{s_2}, \dots, t_{s_m}$ 按顺序连接起来，与将它们底部的字符串 $b_{s_1}, b_{s_2}, \dots, b_{s_m}$ 连接起来的结果完全相同？即：

$$t_{s_1} t_{s_2} \dots t_{s_m} = b_{s_1} b_{s_2} \dots b_{s_m}.$$

如果存在这样的序列，则称 P 有解 (match)。

形式化语言：定义语言 $L_{PCP} = \{\langle P \rangle : P \text{ 有解}\}$ 。其中 $\langle P \rangle$ 是 PCP 实例 P 的某种编码。

变体：**MPCP (Modified PCP)** MPCP 要求任何解序列必须从指定的第一张骨牌开始。可以证明 L_{MPCP} 也是不可判定的，并且可以通过一个技巧将 MPCP 实例转换为等价的 PCP 实例。

PCP 的不可判定性（本部分难度较大）

- 关键定理（Post, 1946）： L_{PCP} 是不可判定的。证明方法仍然是归约。
- 证明思路概述：
 1. 给定一个图灵机 M 和输入 w ，构造一个 PCP 实例 $P = P(M, w)$ 。
 2. 构造使得 M 接受 w 当且仅当 P 有解。
 3. 因为 L_{accept} 不可判定，所以 L_{PCP} 也不可判定。

标准归约证明：这里用 MPCP（要求解必须以指定首骨牌开头）作为中间站，从 L_{accept} 归约到 MPCP，再用“首尾加标记”技巧把 MPCP 转成等价的 PCP。以下直接描述从 (M, w) 构造 MPCP 实例的要点。

1. 配置编码：用分隔符 # 把整条纸带和状态一起编码成字符串，例如 $\$u \ q \ v\$$ 代表纸带内容为 uv ，读头在最左的 v 处、当前状态为 q 。（按照约定，状态符号 $\$q\$$ 右侧紧邻的第一个字符就是读写头当前指向的纸带单元，这样不仅记录了状态，也顺便记录了读写头位置）一条完整的计算历史写成 $\#C_0\#C_1\#\dots\#C_m\#$ ，其中 C_0 是初始配置， C_m 是某个配置（希望它是接受配置）。
2. 目标：让 MPCP 解串拼成一条“上下相同”的历史：上行是 $\#C_0\#C_1\#\dots\#C_m\#$ ，下行也是同样的字符串。这样只有当存在合法的接受计算时才可能匹配成功。
3. 骨牌类型：
 - 启动骨牌：强制第一张骨牌产生 $\#C_0$ 的开头，把求解序列锚定在初始配置上。
 - 复制骨牌：为每个符号（包括 #、纸带符号和状态符号）放入形如 $[a, a]$ 的骨牌，保证合法历史中的大部分字符可以逐个复制到上下两行。
 - 转移骨牌（核心）：对于每条转移规则 $qa \rightarrow bp$ （向右移动）或 $qa \rightarrow bp$ （向左移动）分别设计骨牌，确保只有当相邻的两个配置满足该转移关系时，上下行的片段才能对齐。例如向右移动时，局部片段从 $q \ a \ c$ 变到 $b \ p \ c$ ，就放入骨牌 $[q \ a, b \ p]$ （并配合复制骨牌处理两侧未变的符号）。类似地处理向左移动的情况，需要覆盖被读头跨过的左侧符号。
 - 接受收尾骨牌：一旦出现接受状态 q_{accept} ，放入若干骨牌把剩余部分“吃完”，让上下行都能在接受配置后对齐结束（通常用若干空耗骨牌把尾部同步消掉）。
4. 正确性直观：
 - 如果 M 接受 w ，则存在合法的配置序列。从左到右依次用启动骨牌产生 $\#C_0$ ，然后用复制骨牌逐个输出 C_0 ，遇到相邻配置处使用对应的转移骨牌“跨越”状态变化，再继续用复制骨牌输出其余符号。重复这一过程就能把整段历史拼在上下两行，实现精确匹配。
 - 反之，如果 MPCP 有解，上下行必然组成一条形如 $\#C_0\#\dots\#C_m\#$ 的串。因为只有转移骨牌能跨越状态符号的变化，解串对应的相邻配置必须满足某条真实的转移规则；因此整串必然是一条合法的运行历史，最后一段必须包含接受状态，否则无法用收尾骨牌对齐结束。所以 M 必然在某步接受 w 。
5. 从 MPCP 到 PCP：把每张骨牌上下同时前置一个统一的标记（如 *），并在第一张骨牌的上行再加一个专属标记，确保任意 PCP 解串也必须以同一张骨牌开头。这给出了 $P(M, w)$ ，满足 M 接受 w 当且仅当 $P(M, w)$ 有解。

综上， $L_{accept} \leq_m L_{MPCP} \leq_m L_{PCP}$ ，因此 PCP 不可判定。

字母表大小的影响

- 一般结论：对于任意有限字母表 Σ ($|\Sigma| \geq 2$)，PCP 都是不可判定的。
- 特殊情况：当字母表是单字母表（即 $\Sigma = \{1\}$ ）时，PCP 变成可判定的。因为此时字符串的比较简化为比较它们的长度。

9.2 哥德尔第一不完备定理

背景：佩亚诺算术（Peano Arithmetic, PA）是一个形式系统，其语言包含符号 0（零）、 S （后继函数）、 $+$ （加法）、 $\times$ （乘法）、 $=$ （等于）以及逻辑连接词和量词（ $\neg, \vee, \wedge, \rightarrow, \forall, \exists$ ）。PA 试图公理化自然数的所有性质。

定理内容（哥德尔第一不完备定理）：任何一致的、包含足够多算术（如 PA）的形式系统 F 都是不完备的。即，存在一个在 F 的语言中可表达的命题 ϕ ，使得 ϕ 和 $\neg\phi$ 在 F 中都不可证明。

深入的内容可见第十课。

9.3 在 Peano 算术中表达数学概念

在 PA 中，我们可以用给定的符号定义许多自然数的性质。例如：

- “ x 整除 y ”： $\text{Divides}(x, y) \equiv (\exists z)(y = x \times z)$ 。
- “ x 是素数”： $\text{Prime}(x) \equiv (x > 1) \wedge (\forall y)(\text{Divides}(y, x) \rightarrow (y = 1 \vee y = x))$ 。其中 $x > 1$ 可以表达为 $(\exists k)(x = S(S(k)))$ 。
- “ p 是孪生素数对中较小的那个”： $\text{TwinPrime}(p) \equiv \text{Prime}(p) \wedge \text{Prime}(S(S(p)))$ 。
- 哥德巴赫猜想： $(\forall x)((\exists k)(x = S(S(k))) \rightarrow (\exists y)(\exists z)(\text{Prime}(y) \wedge \text{Prime}(z) \wedge x = y + z))$ 。

通过这些定义，我们可以将许多数学猜想形式化为 PA 中的一个句子。

二、作业题目详细讲解

题目 1

题目

证明在单字母表 $\Sigma = \{1\}$ 上，Post Correspondence Problem 是可判定的。

讲解

我们需要证明，当字母表中只有一个符号（例如 1）时，判定一个 PCP 实例是否有解是可行的。

核心思路：在单字母表上，每个骨牌 $[t_i, b_i]$ 中的 t_i 和 b_i 都是由一串 1 组成的字符串。它们之间的区别仅在于长度。设 $|t_i|$ 表示 t_i 中 1 的个数（即长度）， $|b_i|$ 同理。

此时，PCP 有解意味着存在一个非空的下标序列 $i_1, i_2, \dots, i_m$ ，使得：

$$t_{i_1} t_{i_2} \dots t_{i_m} = b_{i_1} b_{i_2} \dots b_{i_m}$$

由于所有字符都是 1，这个等式等价于两边字符串的长度相等：

$$|t_{i_1}| + |t_{i_2}| + \dots + |t_{i_m}| = |b_{i_1}| + |b_{i_2}| + \dots + |b_{i_m}|.$$

定义差值 $d_i = |t_i| - |b_i|$ 。那么上面的等式就变成了：

$$d_{i_1} + d_{i_2} + \cdots + d_{i_m} = 0.$$

原问题转化为：是否存在一组非负整数 $c_1, c_2, \dots, c_k$ （其中 c_i 表示第 i 种骨牌被选用的次数，且至少有一个 $c_i > 0$ ），使得

$$c_1 d_1 + c_2 d_2 + \cdots + c_k d_k = 0.$$

这是一个整数线性丢番图方程。

如何判定：这类方程是否有非负整数解（且不全为零）是可判定的。判定方法有很多，例如考虑正负性来证明（有零必可行，正负性全相同必不行，有正有负必可行）。

结论：由于在单字母表上，PCP 问题可以归约为一个可判定的整数方程求解问题，所以单字母表上的 PCP 是可判定的。

题目 2

题目

证明在二进制字母表 $\Sigma = \{0, 1\}$ 上，Post Correspondence Problem 是不可判定的，假设在任意有限字母表上 PCP 是不可判定的。

讲解

已知结论：对于任意有限字母表（大小至少为 2），PCP 是不可判定的。我们需要证明，即使限制字母表为 $\{0, 1\}$ ，PCP 仍然不可判定。

证明思路：使用归约。我们将任意字母表 Γ 上的一个 PCP 实例 P ，转化为一个二进制字母表 $\{0, 1\}$ 上的实例 P' ，并且保持“有解性”不变。如果二进制 PCP 可判定，那么通过这个转化，我们就能判定任意字母表上的 PCP，这与已知矛盾。

详细构造：

1. 设 P 是字母表 Γ 上的一个 PCP 实例，包含骨牌 $[t_1, b_1], \dots, [t_k, b_k]$ ，其中 $t_i, b_i \in \Gamma^*$ 。
2. 因为 Γ 是有限集，我们可以为 Γ 中的每个符号分配一个唯一的、固定长度的二进制码字。（例如若 $|\Gamma| = m$ ，则我们可以用 $\lceil \log_2 m \rceil$ 位二进制数来表示每个符号）简单的方法是，直接定义一个单射的编码函数 $f: \Gamma \rightarrow \{0, 1\}^+$ ，要求 f 是一个同态，即对于任何字符串 $u, v \in \Gamma^*$ ，有 $f(uv) = f(u)f(v)$ 。这可以通过为每个符号分配一个互不相同的二进制串，并确保没有一个串是另一个串的前缀（前缀码）来实现，例如使用 Huffman 编码的思想。（但是如果字母表中只有一个字母，霍夫曼编码显然无法区分，因此就不行）
3. 应用这个编码函数到 P 的每个骨牌上，得到新的骨牌集合 P' ：

$$P' = \{[f(t_1), f(b_1)], [f(t_2), f(b_2)], \dots, [f(t_k), f(b_k)]\}.$$

显然， P' 是一个在字母表 $\{0, 1\}$ 上的 PCP 实例。

4. 等价性证明：

- 如果 P 有解，即存在序列 $i_1, \dots, i_m$ 使得 $t_{i_1} \dots t_{i_m} = b_{i_1} \dots b_{i_m}$ 。由于 f 是同态，两边应用 f 得到：

$$f(t_{i_1}) \dots f(t_{i_m}) = f(b_{i_1}) \dots f(b_{i_m}).$$

这正是 P' 的一个解。所以 P' 有解。

- 反之，如果 P' 有解，即存在序列使得 $f(t_{i_1}) \dots f(t_{i_m}) = f(b_{i_1}) \dots f(b_{i_m})$ 。由于 f 是单射的同态（且解码是唯一的），我们可以“解码”回原字符串，得到 $t_{i_1} \dots t_{i_m} = b_{i_1} \dots b_{i_m}$ 。所以 P 有解。因此， P 有解当且仅当 P' 有解。

推出矛盾：假设在 $\{0, 1\}$ 上的 PCP 是可判定的。那么，对于任意字母表 Γ 上的 PCP 实例 P ，我们可以先按上述方法构造出对应的二进制实例 P' ，然后用这个假定的判定器去判断 P' 。根据等价性，我们就知道了 P 是否有解。这意味着我们有了一个判定任意字母表上 PCP 的算法，与“任意字母表上 PCP 不可判定”的已知结论矛盾。所以，在二进制字母表 $\{0, 1\}$ 上，PCP 也是不可判定的。

题目 3

题目

在 Peano 算术中，使用符号 $0, S, +, \times, =$ 和量词 $\forall, \exists$ 以及逻辑连接词表达双素数猜想。

- 定义公式 $\text{Prime}(x)$ 表示 x 是素数。
- 定义公式 $\text{TwinPrime}(p)$ 表示 p 是双素数对中较小的那个数。
- 写一个单一的一阶 PA 公式，表示：对于每个数 x ，存在一个更大的双素数 $p > x$ 。

讲解

我们需要在 PA 的框架内，仅使用允许的符号来形式化定义这些概念。注意，PA 中没有直接的关系符号 $>$ ，但我们可以用 $=$ 和 $+$ 来定义它： $a > b$ 可以定义为 $(\exists k)(a = b + S(k))$ 。

(a) 定义 $\text{Prime}(x)$

素数的定义是：大于 1 的自然数，并且只能被 1 和它自身整除。

- “大于 1”： $x > 1$ 可以写为 $(\exists k)(x = S(S(k)))$ 。因为 $S(S(k)) = k + 2$ ，这覆盖了所有 $x \geq 2$ 。
- “ y 整除 x ”： $\text{Divides}(y, x) \equiv (\exists z)(x = y \times z)$ 。
- “ y 是平凡除数（1 或 x ）”： $(y = S(0)) \vee (y = x)$ 。注意 $S(0)$ 表示数字 1。
- 合起来： x 是素数当且仅当 $x > 1$ ，并且任何能整除 x 的 y 都必须是平凡除数。

$$\text{Prime}(x) \equiv (\exists k)(x = S(S(k))) \wedge (\forall y)(\text{Divides}(y, x) \rightarrow (y = S(0) \vee y = x)).$$

(b) 定义 $\text{TwinPrime}(p)$

双素数对是指一对素数 $(p, p + 2)$ 。所以 p 是其中较小的那个，当且仅当 p 和 $p + 2$ 都是素数。

- “ $p+2$ ”：在 PA 中， $p + 2$ 可以表示为 $S(S(p))$ 。
- 因此：

$$\text{TwinPrime}(p) \equiv \text{Prime}(p) \wedge \text{Prime}(S(S(p))).$$

(c) 表达“对每个 x ，存在更大的双素数 p ”

我们需要一个公式 ϕ ，使得 ϕ 为真当且仅当双素数猜想成立。双素数猜想断言：存在无穷多个双素数对。这等价于说，对于任何自然数 x ，总存在一个双素数 p 大于 x 。

1. “ $p > x$ ”: $(\exists k)(p = x + S(k))$ 。这里 $S(k)$ 是某个正数, 确保 $p > x$ 。

2. 合起来:

$$\phi \equiv (\forall x)(\exists p)(\exists k)(p = x + S(k) \wedge \text{TwinPrime}(p)).$$

这个公式读作: 对于所有 x , 存在 p 和 k , 使得 $p = x + (k + 1)$ 并且 p 是双素数 (较小的那个)。由于 k 可以取任何自然数, p 就可以取任何大于 x 的数。

最终答案:

(a) $\text{Prime}(x) \equiv (\exists k)(x = S(S(k))) \wedge (\forall y)((\exists z)(x = y \times z) \rightarrow (y = S(0) \vee y = x))$ 。

(b) $\text{TwinPrime}(p) \equiv \text{Prime}(p) \wedge \text{Prime}(S(S(p)))$ 。

(c) $(\forall x)(\exists p)(\exists k)(p = x + S(k) \wedge \text{TwinPrime}(p))$ 。

第十节课复习资料

一、本节课知识点讲解

10.1 哥德尔第一不完备定理回顾

在第九节课, 我们学习了 Post Correspondence Problem (PCP) 的不可判定性。哥德尔第一不完备定理的证明思想与之紧密相关:

- 定理: 任何一致的、包含足够算术 (如 Peano 算术) 的形式系统 F 都是不完备的。即存在一个在 F 中可表达的命题 φ , 使得 φ 和 $\neg\varphi$ 在 F 中都不可证明。
- 证明思路 (利用 PCP):
 1. 在形式系统 F 中, 我们可以表达类似 “PCP 实例 P 有解” 这样的命题。
 2. 如果 F 是完备的 (每个命题或其否定都可证), 那么对于任何 PCP 实例 P , F 总能证明 “ P 有解” 或 “ P 无解”。
 3. 这将给出一个判定 PCP 的算法 (在 F 中系统性地枚举所有证明, 直到找到关于 P 的证明), 这与 PCP 的不可判定性矛盾。
 4. 因此假设错误, F 必然是不完备的。

核心: 任何足够强大的形式系统, 其 “可证明性” 概念本身可以在系统内表述, 从而构造出自我指涉的命题, 导致无法证明其自身的一致性。

10.2 计算复杂性理论入门

计算理论不仅关心问题 “是否能计算” (可判定性), 也关心 “需要多少资源来计算” (复杂性)。我们主要关注时间和空间资源。

- **DTIME(T(n))**: 所有能被一个 (确定型) 多带图灵机在 $O(T(n))$ 时间内判定的语言的集合。
- **P 类**: 多项式时间可判定的问题的集合。

$$P = \bigcup_{k \geq 1} \text{DTIME}(n^k)$$

直观上, P 类代表了现实中 “易解” 或 “高效可解” 的问题。

- 其他复杂性类（简要了解）：
 - **NP**：多项式时间内可验证（给定一个“证书”或“解”）。
 - **PSPACE**：多项式空间内可判定。
 - **EXP**：指数时间（ $2^{n^{O(1)}}$ ）内可判定。
- 主要开放问题：
 - $P \stackrel{?}{=} NP$ （是否所有易验证的问题都易解）
 - $P \stackrel{?}{=} BPP$ （是否能在多项式时间内得到概率正确答案的问题，都可以被一个纯粹的确定性算法在多项式时间内解决）

10.3 归约（Reduction）

归约是比较问题难度的核心工具。如果问题 A 可以“转化”为问题 B，并且这个转化本身不太难，那么解决 B 的能力就隐含了解决 A 的能力。我们说“A 归约到 B”。

- 多一归约 (**Many-one Reduction**, $\leq_m$): 存在一个可计算函数 f ，使得对于所有输入 x ， $x \in A \iff f(x) \in B$ 。直观：要判断 x 是否属于 A，只需计算 $f(x)$ 并判断它是否属于 B。
- 图灵归约 (**Turing Reduction**, $\leq_T$): 存在一个拥有预言 (**Oracle**) 的图灵机，它可以将问题 B 的判定当作一个“黑盒子”子程序（调用一次或多次）来使用，并最终判定问题 A。
 - 预言图灵机：除了普通纸带，还有一条特殊的“预言带”。当机器进入特殊状态时，能在一步之内知道“当前预言带上的字符串是否属于预言语言 B”（或者得到某个函数 f 的值）。
 - 图灵归约比多一归约更灵活，因为它允许调用预言多次，并且不要求转化是函数形式。通过假设“询问预言机”只需一步，研究者可以将注意力集中在主程序如何利用这个答案来解决问题，而不需要关心预言机内部是如何复杂地得出结论的。
- 多项式时间归约：
 - 卡普归约 (**Karp Reduction**, $\leq_P$): 多一归约，且归约函数 f 是多项式时间可计算的。
 - 库克归约 (**Cook Reduction**, $\leq_P^T$): 图灵归约，且整个预言图灵机的运行时间是多项式时间（不计预言调用的内部时间，只计调用次数和外部步骤）。
 - 之所以引入这两种新的归约，是因为有时我们也会关注归约的时间，如果一种问题可被归约到简单问题，但耗时太久，它可能也并不能说一定简单。

归约的性质：

1. 自反性： $L \leq_m L$ ， $L \leq_T L$ 。
2. 传递性：如果 $L_1 \leq_m L_2$ 且 $L_2 \leq_m L_3$ ，则 $L_1 \leq_m L_3$ （图灵归约同理）。
3. 关键用途：如果 $L_1 \leq_m L_2$ 且 L_2 可判定，则 L_1 可判定。反之，如果 L_1 不可判定，则 L_2 也不可判定。这对于证明问题不可判定至关重要。
4. 多一归约是图灵归约的特例： $L_1 \leq_m L_2 \Rightarrow L_1 \leq_T L_2$ 。

二、作业题目详细讲解

题目 1

题目

使用一阶 Peano 算术公式表达 “ $m = 2^n$ ”。

讲解

我们需要在 Peano 算术 (PA) 的语言中构造一个公式 $\phi(m, n)$, 使其为真当且仅当 m 等于 2 的 n 次幂。PA 的语言非常基础, 只有常量 0, 后继函数 S , 加法 $+$, 乘法 $\times$, 等于 $=$, 以及逻辑连接词和量词。它没有内置的指数运算。

为了表达 2^n , 我们需要描述一个序列: $a_0 = 2^0 = 1, a_1 = 2^1 = 2, a_2 = 4, \dots, a_n = 2^n = m$ 。在 PA 中直接谈论“序列”是不允许的。哥德尔引入的 β 函数编码技巧解决了这个问题。

β 函数的作用: 它用两个数字 a 和 b 作为“密钥”, 可以编码一个有限序列 $a_0, a_1, \dots, a_n$ 。表达式 $\beta(a, b, i) = r$ 在 PA 中的含义是: 由 a, b 编码的序列的第 i 项是 r 。

β 函数在 PA 中是可以定义的 (虽然较复杂):

$$\beta(a, b, i) = r \quad \text{定义为} \quad \exists q (a = q \times (1 + S(i) \times b) + r \wedge r < 1 + S(i) \times b)$$

这里 $S(i)$ 就是 $i + 1$ 。这个定义利用了中国剩余定理的思想: 通过选择足够大的、两两互素的模数 $(1 + (i + 1) \times b)$, 可以确保每个余数 r (即序列元素) 能唯一地被定位和提取。条件 $r < 1 + S(i) \times b$ 保证了 r 是那个唯一的余数。

构造公式 $\phi(m, n)$: 利用 β 函数, 我们可以描述前面提到的序列:

1. 序列第一项是 1: $\beta(a, b, 0) = S(0)$ 。($S(0)$ 表示数字 1)
2. 序列满足递推关系: 每一项是前一项乘以 2。对于所有 k , 如果 $k < n$, 那么第 $k + 1$ 项等于第 k 项乘以 2。

$$\forall k (k < n \supset \beta(a, b, S(k)) = \beta(a, b, k) \times S(S(0)))$$

$S(S(0))$ 表示数字 2。这里的蕴含符号就是推出的意思, 和箭头等价。

3. 序列的第 n 项等于 m : $\beta(a, b, n) = m$ 。

将这三部分用“且”连接起来, 并声明存在这样的编码 a 和 b , 就得到了完整的公式:

$$\phi(m, n) \equiv \exists a \exists b \left[\beta(a, b, 0) = S(0) \wedge \forall k (k < n \supset \beta(a, b, S(k)) = \beta(a, b, k) \times S(S(0))) \wedge \beta(a, b, n) = m \right]$$

解读: 这个公式说: “存在两个数 a 和 b , 它们编码了一个序列。这个序列的第一个数是 1, 并且序列中每个后面的数都是前一个数的两倍, 直到第 n 个数, 而这个数正好是 m 。”这正是 $m = 2^n$ 的定义。

小结: 在 PA 中表达指数运算, 必须借助序列编码 (如 β 函数) 来绕过其语言的局限性。虽然公式看起来复杂, 但这是该框架下的标准且严谨的方法。

题目 2

题目

证明 P 在 Kleene 星号下封闭。换言之，如果语言 L 属于 P ，那么 L^* 也属于 P 。

讲解

已知 $L \in P$ ，即存在一个（确定型）图灵机 M 在多项式时间（比如 $O(n^c)$ ）内判定 L 。我们需要证明 $L^* \in P$ ，即要构造一个多项式时间的图灵机 M^* 来判定 L^* 。

判定思路——动态规划：我们采用自底向上的动态规划来判断输入字符串 w （长度为 n ）是否可以被这样分割。

1. 定义状态：令 $dp[i]$ 是一个布尔值，表示 w 的前 i 个字符组成的前缀 $w[1..i]$ 是否属于 L^* 。这里 $w[1..i]$ 表示从第一个字符到第 i 个字符的子串。我们规定 $dp[0] = \text{true}$ ，因为空前缀（即空串）总是属于 L^* 。
2. 状态转移：要计算 $dp[i]$ （ $1 \leq i \leq n$ ），我们需要检查是否存在一个“断点” j （ $0 \leq j < i$ ），使得：
 - 前缀 $w[1..j]$ 已经可以分割成 L 中的串（即 $dp[j] = \text{true}$ ）。
 - 并且从第 $j+1$ 到第 i 个字符组成的子串 $w[j+1..i]$ 本身属于 L 。如果存在这样的 j ，那么 $w[1..i]$ 就可以由 $w[1..j]$ （属于 L^* ）再连接上一个属于 L 的串 $w[j+1..i]$ 得到，因此 $dp[i] = \text{true}$ 。
 用公式表示：

$$dp[i] = \bigvee_{j=0}^{i-1} (dp[j] \wedge (w[j+1..i] \in L))$$

3. 最终答案：如果 $dp[n] = \text{true}$ ，则整个字符串 w 属于 L^* ；否则不属于。

构造判定机 M^* ：输入：字符串 w ，长度 n 。

1. 初始化数组 $dp[0..n]$ ，设 $dp[0] = \text{true}$ ，其余为 false 。
2. 对于 i 从 1 到 n ：
 - 对于 j 从 0 到 $i-1$ ：
 - 如果 $dp[j] == \text{true}$ ，则检查子串 $w[j+1..i]$ 是否属于 L 。这可以通过模拟已知的判定机 M 在输入 $w[j+1..i]$ 上来完成。
 - 如果 M 接受该子串，则设置 $dp[i] = \text{true}$ 并跳出内层循环（无需检查更多的 j ）。
3. 如果 $dp[n] == \text{true}$ ，则 M^* 接受 w ；否则拒绝。

时间复杂度分析：

- 外层循环 i 有 n 次。内层循环 j 也最多有 n 次。
- 在最内层，我们需要模拟 M 来检查一个长度最多为 n 的子串是否属于 L 。因为 M 是多项式时间的，设其运行时间为 $O(l^c)$ ，其中 l 是输入长度。对于长度为 l 的子串，一次模拟需要 $O(n^c)$ 时间（因为 $l \leq n$ ）。
- 因此，总时间开销为 $O(n) \times O(n) \times O(n^c) = O(n^{c+2})$ 。这是一个关于 n 的多项式。

结论：我们成功构造了一个在多项式时间内判定 L^* 的图灵机 M^* ，因此 $L^* \in P$ 。

题目 3

题目

令

$$L_1 = \{(\alpha, x) : \text{图灵机 } M_\alpha \text{ 接受输入 } x\},$$

和

$$L_2 = \{\alpha : \text{图灵机 } M_\alpha \text{ 在空输入上停机}\}.$$

证明 $L_1 \leq_T L_2$ 。

讲解

我们需要证明 L_1 可以图灵归约到 L_2 。即，存在一个拥有 L_2 预言（Oracle）的图灵机，能够判定 L_1 。

理解图灵归约：判定 L_1 的机器可以“询问”一个神奇的 L_2 黑盒子：“字符串 β 是否属于 L_2 ？”并立刻得到是或否的答案。我们的任务就是设计一个算法，它对于任意输入 (α, x) ，通过提出一个（或多个）巧妙构造的关于 L_2 的问题，最终判断出 (α, x) 是否属于 L_1 。

核心构造思路：我们要判断的是： M_α 是否接受 x ？我们可以构造一个新的图灵机 N ，它的行为被精心设计，使得 N 在空输入上的停机行为等价于 M_α 对 x 的接受行为。

构造图灵机 N ：

- 输入： N 在空输入上运行（即输入带是空的）。
- 行为：
 1. N 模拟图灵机 M_α 在输入 x 上的运行。
 2. 在模拟过程中：
 - 如果 M_α 最终进入接受状态，那么 N 就停机并接受（即进入自己的接受状态）。
 - 如果 M_α 最终进入拒绝状态，那么 N 就故意进入一个无限循环。
 - 如果 M_α 本身就在 x 上永不停机（无限循环），那么 N 的模拟也将永远进行下去，即 N 也永不停机。
- 关键点： N 在空输入上停机当且仅当 M_α 在输入 x 上接受。

利用 L_2 的预言：现在，假设我们有 L_2 的预言。 L_2 的定义是： $\alpha \in L_2$ 当且仅当 M_α 在空输入上停机。我们构造的 N 也是一个图灵机，它有一个编码，记作 α' 。注意， α' 的构造依赖于给定的 (α, x) （从 (α, x) 生成 α' 是一个可计算的过程）。

根据 L_2 的定义和 N 的行为，我们有：

$$\alpha' \in L_2 \iff N \text{ 在空输入上停机} \iff M_\alpha \text{ 接受 } x \iff (\alpha, x) \in L_1$$

判定 L_1 的 Oracle 图灵机算法：输入： (α, x) 。

1. 根据 (α, x) ，构造出上述图灵机 N 的编码 α' 。
2. 向 L_2 预言提问：“ α' 是否属于 L_2 ？”

3. 如果预言回答“是”，则我们判定 $(\alpha, x) \in L_1$ ，接受。
4. 如果预言回答“否”，则我们判定 $(\alpha, x) \notin L_1$ ，拒绝。

因为整个过程中，从输入到提出预言问题的计算是有限的，并且只进行了一次预言调用，所以我们确实构建了一个以 L_2 为预言的图灵机来判定 L_1 。这符合图灵归约 $L_1 \leq_T L_2$ 的定义。

总结：这个证明展示了图灵归约的强大灵活性。我们通过构造一个辅助机器 N ，将对 L_1 的判定（ M_α 是否接受 x ）转化为了对 L_2 的单次查询（ N 的编码是否在 L_2 中）。这是解决此类问题的典型手法。